BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



TIỂU LUẬN MÔN HỌC
PHÂN TÍCH DỮ LIỆU LỚN

XÂY DỰNG HỆ THỐNG PHÂN TÍCH DỮ LIỆU CẢM BIẾN ĐIỆN THEO THỜI GIAN THỰC ĐỂ DỰ BÁO NHU CẦU TIÊU THỤ VÀ CẢNH BÁO BẤT THƯỜNG TRONG TÒA NHÀ THÔNG MINH

Giáo viên hướng dẫn: Pgs. Nguyễn Mạnh Hùng
Nhóm thực hiện:
Mã lớp học: BDAN437764_01

TP. Hồ Chí Minh, tháng năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



TIỂU LUẬN MÔN HỌC
PHÂN TÍCH DỮ LIỆU LỚN

XÂY DỰNG HỆ THỐNG PHÂN TÍCH DỮ LIỆU CẢM BIẾN ĐIỆN THEO THỜI GIAN THỰC ĐỂ DỰ BÁO NHU CẦU TIÊU THỤ VÀ CẢNH BÁO BẤT THƯỜNG TRONG TÒA NHÀ THÔNG MINH

GVHD: Pgs. Nguyễn Mạnh Hùng

SVTH: Nhóm

1. Hoàng Ngọc Dung 23139006
2. Hoàng Mạnh Hùng 23139017
3. Trần Hữu Dương 23139009
4. Phạm Quang Phúc 23139034

TP. Hồ Chí Minh, tháng năm 2026

DANH SÁCH NHÓM VIẾT BÁO CÁO CUỐI KỲ

HỌC KỲ 1 NĂM HỌC 2026–2027

1. **Mã lớp môn học:** BDAN437764_01
2. **Giảng viên hướng dẫn** :Pgs. Nguyễn Mạnh Hùng
3. **Tên đề tài:** Xây dựng hệ thống phân tích dữ liệu cảm biến điện theo thời gian thực để dự báo nhu cầu tiêu thụ và cảnh báo bất thường trong tòa nhà thông minh.
4. **Danh sách nhóm và phân công nhiệm vụ**

STT	Họ và tên sinh viên	MSSV	Nội dung thực hiện	Ký tên
1	Hoàng Ngọc Dung	23139006	Phân tích dữ liệu, xây dựng Pipeline ETL (Bronze - Silver - Gold) bằng Spark.	
2	Hoàng Mạnh Hùng	23139024	Xây dựng phân tích pipline (có chế cảnh báo bất thường) và kiểm thử	
3	Phạm Quang Phúc	23139034	Cấu hình luồng Kafka Streaming, chạy Pilot và kiểm thử hiệu năng toàn hệ thống.	
4	Trần Hữu Dương	23139009	Xây dựng mô hình Random Forest (Spark MLlib) và cơ chế cảnh báo bất thường.	

Nhận xét của giáng viên

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Ngày …. tháng …. năm 2026

Giảng viên chấm điểm

LỜI CAM ĐOAN

Chúng em xin cam đoan đề tài nghiên cứu là trung thực. Những kết quả số liệu, thông tin phục vụ cho quá trình xử lý và hoàn thành bài nghiên cứu đều được thu thập từ các nguồn khác nhau, có ghi rõ nguồn gốc. Tất cả các thông tin số liệu, bảng, hình ảnh và kết quả nghiên cứu được đều được trình bày một cách minh bạch và chân thực, không chứa bất kỳ sự biến đổi hay chỉnh sửa nào có thể làm ảnh hưởng đến tính chính xác của kết quả.

Các nguồn dữ liệu và thông tin được sử dụng trong nghiên cứu đã được thu thập từ các nguồn đáng tin cậy và đã được ghi rõ nguồn gốc. Bất kỳ tài liệu tham khảo, nghiên cứu trước đây hoặc ý kiến cá nhân được sử dụng đều đã được đặc tả một cách rõ ràng để đảm bảo tính minh bạch và khách quan. Chúng em xin chịu hoàn toàn trách nhiệm, kỷ luật của bộ môn và nhà trường nếu như có bất cứ vấn đề nào.

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến Pgs. Nguyễn Mạnh Hùng giảng viên bộ môn Phân tích dữ liệu lớn. Trong suốt quá trình học tập, Thầy đã rất tâm huyết dạy và hướng dẫn cho chúng tôi nhiều điều bổ ích trong môn học và kỹ năng làm một bài nghiên cứu để chúng tôi có đủ kiến thức thực hiện bài báo cáo này.

Tuy nhiên vì kiến thức bản thân còn nhiều hạn chế và sự tìm hiểu chưa sâu sắc nên không tránh khỏi những thiếu sót. Mong Thầy sẽ châm chước và cho chúng em những lời góp ý để bài nghiên cứu của chúng tôi sẽ hoàn thiện hơn. Một lần nữa, chúng tôi xin gửi lời cảm ơn sâu sắc đến Thầy và chúc Thầy luôn mạnh khỏe, hạnh phúc và thành công trong sự nghiệp.

TP. Hồ Chí Minh, tháng năm 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI 1
1. Tình hình nghiên cứu 1
2. Đặt vấn đề 3
3. Mục tiêu của đề tài 4
4. Nội dung nghiên cứu 5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 6
2.1. Tổng quan về dữ liệu tiêu thụ điện theo chuỗi thời gian 6
2.1.1 Dữ liệu cảm biến và công tơ năng lượng 6
2.1.2 Dữ liệu chuỗi thời gian 7
2.1.3 Đặc điểm dữ liệu 7
2.1.4 Các vấn đề chất lượng dữ liệu cảm biến 8
2.2. Xử lý dữ liệu lớn và dữ liệu thời gian thực 9
2.2.1. Xử lý theo lô và xử lý luồng 9
2.2.3. Batch Processing và Stream Processing 9
2.2.5 Kiến trúc Kappa 9
2.4. Lưu trữ dữ liệu 9
2.4.1. Delta Lake và kiến trúc Bronze – Silver – Gold 9
2.5 Học máy và mô hình ước lượng tiêu thụ điện 10
2.5.1 Xác định bài toán hồi quy 10
2.5.2 Nguyên lý Random Forest Regression 11
2.5.3 Triển khai bằng Spark MLlib 11
2.6 Sai số dự báo, phát hiện bất thường và chỉ số đánh giá 12
2.6.1 Sai số dự báo và mức độ lệch 12
2.6.2 Chỉ số đánh giá hồi quy 12
2.6.3 Chọn ngưỡng bất thường 13
2.6.4 Precision, Recall và F1 score 13
CHƯƠNG 3: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG 14
3.1. Phân tích yêu cầu và kiến trúc tổng thể hệ thống 14
3.1.1. Yêu cầu chức năng và phi chức năng 14
3.1.2. Kiến trúc tổng thể 16

3.1.3. Luồng dữ liệu trong hệ thống 17
3.2. Chuẩn bị và mô phỏng dữ liệu thời gian thực 19
3.2.1. Lựa chọn và tiền xử lý dữ liệu BDG2 19
3.2.2. Xây dựng Python Producer 19
3.2.3. Định dạng và cơ chế phát dữ liệu 19
3.3. Xây dựng luồng xử lý dữ liệu thời gian thực 19
3.3.1. Cấu hình Apache Kafka 19
3.3.2. Kết nối Kafka với Spark Structured Streaming 19
3.3.3. Làm sạch và xử lý dữ liệu luồng 19
3.3.4. Xử lý dữ liệu đến trễ và dữ liệu trùng 19
3.4. Xây dựng hệ thống lưu trữ dữ liệu 19
3.4.1. Tầng Bronze 19
3.4.2. Tầng Silver 21
3.4.3. Tầng Gold 22
3.5. Xây dựng mô hình dự báo tiêu thụ điện 24
3.5.1 Tập dữ liệu và cách chia theo thời gian 24
3.5.2. Đặc trưng và cấu hình huấn luyện 24
3.6 Phát hiện bất thường 25
3.7 Thiết kế kịch bản kiểm thử và chỉ số đánh giá hệ thống 27
3.7.1 Kiểm thử chất lượng phân tích 27
3.7.2 Kiểm thử theo tải và dữ liệu lỗi 27
3.7.3 Quy ước đo và thu thập log 27
CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ 28
4.1. Dữ liệu và cấu hình phát thử nghiệm 28
4.2. Kết quả ETL và lưu trữ dữ liệu 29
4.2.1. Kết quả chuyển dữ liệu RAW → Bronze 29
4.2.2. Kết quả làm sạch Bronze → Silver 30
4.2.3. Kết quả tổng hợp Silver → Gold 31
4.2.4. So sánh dung lượng CSV và Parquet 32
4.2.5. So sánh thời gian đọc, ghi và xử lý ETL 33
4.3. Kết quả Kafka và Python Producer 34

4.3.1. Định dạng bản tin và kiểm tra topic 34
4.3.2. Thông lượng Producer theo tốc độ phát 34
4.3.3. Ảnh hưởng của batch size và số partition 35
4.3.4. Độ ổn định và tỷ lệ gửi thành công 35
4.5 Kết quả mô hình dự báo tiêu thụ điện 35
4.5.1 Kết quả huấn luyện Random Forest Regression 35
4.5.2 Đánh giá bằng MAE, RMSE và R^2 36
4.5.3 So sánh giá trị dự báo với giá trị thực tế 38
4.5.4 Phân tích sai số và đặc trưng quan trọng 39
4.6 Kết quả phát hiện bất thường 40
4.6.1 Kết quả tính residual và ngưỡng bất thường 40
4.6.2 Các kịch bản bất thường được thử nghiệm 41
4.6.3 Đánh giá bằng Precision, Recall và F1 score 41
4.6.4 Phân tích các trường hợp phát hiện đúng và sai 42
4.6.5 Minh họa quy trình giám sát hoàn chỉnh 44
4.8 Đánh giá hiệu năng toàn hệ thống 45
4.8.1 Benchmark theo nhiều mức tải 45
4.8.2 Throughput toàn hệ thống 46
4.8.3 End to End Latency 47
4.8.4 Backlog và khả năng bắt kịp dữ liệu 49
4.8.5 Điểm nghẽn, khả năng mở rộng và nhận xét chung 50
KẾT LUẬN 53
TÀI LIỆU THAM KHẢO 54

DANH MỤC CÁC CHỮ VIẾT TẮT

ACID	Tính nguyên tử, nhất quán, cô lập và bền vững của giao dịch
AI	Trí tuệ nhân tạo
API	Giao diện lập trình ứng dụng
ASHRAE	Hiệp hội Kỹ sư Sưởi ấm, Làm lạnh và Điều hòa không khí Hoa Kỳ
CPU	Bộ xử lý trung tâm
CSV	Tệp dữ liệu phân tách bằng dấu phẩy
DAG	Đồ thị có hướng không chu trình
ETL	Trích xuất – Biến đổi – Nạp dữ liệu
GB	Gigabyte – đơn vị đo dung lượng dữ liệu
HDFS	Hệ thống tệp phân tán Hadoop
IoT	Internet vạn vật
JSON	Định dạng trao đổi dữ liệu dạng đối tượng JavaScript
MB	Megabyte – đơn vị đo dung lượng dữ liệu
ML	Học máy
NoSQL	Cơ sở dữ liệu phi quan hệ / không chỉ sử dụng SQL
RAM	Bộ nhớ truy cập ngẫu nhiên
RDD	Tập dữ liệu phân tán có khả năng phục hồi
SQL	Ngôn ngữ truy vấn có cấu trúc
WSL	Hệ thống con Windows dành cho Linux
ACID	Tính nguyên tử, nhất quán, và bền vững của giao dịch

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI

1. Tình hình nghiên cứu

Trong những năm gần đây, sự phát triển của công tơ thông minh, hệ thống quản lý tòa nhà và các thiết bị Internet vạn vật đã tạo ra lượng lớn dữ liệu tiêu thụ năng lượng theo thời gian. Khác với phương pháp ghi nhận điện năng định kỳ, các hệ thống hiện đại có khả năng thu thập liên tục dữ liệu từ nhiều công tơ, nhiều loại tòa nhà và nhiều khu vực khác nhau. Nguồn dữ liệu này cho phép người quản lý theo dõi tình trạng sử dụng năng lượng, xây dựng mô hình dự báo và phát hiện sớm các hành vi tiêu thụ bất thường. Tuy nhiên, khi số lượng công tơ và tần suất đo tăng lên, việc lưu trữ và phân tích dữ liệu bằng các công cụ truyền thống trên một máy tính đơn lẻ trở nên kém hiệu quả. Vì vậy, xử lý dữ liệu lớn và xử lý luồng đang trở thành những hướng tiếp cận quan trọng trong lĩnh vực quản lý năng lượng thông minh.

Một trong những nền tảng dữ liệu tiêu biểu phục vụ nghiên cứu trong lĩnh vực này là Building Data Genome Project 2, gọi tắt là BDG2, được Miller và cộng sự công bố năm 2020 [1]. Bộ dữ liệu gồm 3.053 công tơ thuộc 1.636 tòa nhà phi dân cư tại 19 địa điểm ở Bắc Mỹ và châu Âu. Dữ liệu được thu thập theo từng giờ trong hai năm 2016 và 2017, tương ứng khoảng 53,6 triệu phép đo. Các công tơ ghi nhận nhiều loại năng lượng và tài nguyên khác nhau như điện, nước nóng, nước lạnh, hơi nước, khí đốt, năng lượng mặt trời và nước tưới. Ngoài dữ liệu công tơ, BDG2 còn cung cấp thông tin về đặc điểm tòa nhà và điều kiện thời tiết. Quy mô và mức độ đa dạng này giúp BDG2 phù hợp với các nghiên cứu về dự báo năng lượng, so sánh hiệu quả vận hành và phát hiện bất thường.

Một phần dữ liệu BDG2 đã được sử dụng trong cuộc thi ASHRAE Great Energy Predictor III nhằm tìm kiếm các phương pháp dự báo mức tiêu thụ năng lượng cho tòa nhà [2]. Cuộc thi cung cấp hơn 20 triệu bản ghi huấn luyện từ 2.380 công tơ thuộc 1.448 tòa nhà và thu hút 4.370 người tham gia. Kết quả cho thấy các mô hình cây tăng cường và mô hình tổ hợp đạt độ chính xác cao, đồng thời khẳng định rằng quá trình làm sạch dữ liệu và xây dựng đặc trưng có ảnh hưởng lớn đến hiệu quả dự báo. Điều này cho thấy đối với dữ liệu năng lượng tòa nhà, việc lựa chọn mô hình chỉ là một phần của bài toán; chất lượng dữ liệu, cách xử lý giá trị thiếu, thông tin thời gian, thời tiết và đặc điểm của từng tòa nhà cũng đóng vai trò quan trọng.

Bên cạnh dự báo, phát hiện bất thường trong tiêu thụ năng lượng cũng nhận được nhiều sự quan tâm. Himeur và cộng sự đã tổng hợp các phương pháp dựa trên trí tuệ nhân tạo dùng để phát hiện mức tiêu thụ bất thường trong tòa nhà [3]. Các phương pháp phổ biến bao gồm kiểm tra bằng ngưỡng thống kê, phân cụm, mô hình dự báo, mạng nơ-ron và autoencoder. Trong đó, cách tiếp cận dựa trên dự báo xây dựng một mô hình biểu diễn mức tiêu thụ bình thường, sau đó so sánh giá trị dự đoán với giá trị thực tế. Nếu độ lệch giữa hai giá trị vượt quá một ngưỡng cho phép, số đo được xem là một trường hợp cần cảnh báo.

Tuy nhiên, nghiên cứu của Himeur và cộng sự cũng chỉ ra một số khó khăn còn tồn tại, đặc biệt là sự thiếu hụt các bộ dữ liệu được gán nhãn bất thường, sự khác biệt trong định nghĩa thế nào là một hành vi tiêu thụ bất thường và việc chưa có hệ thống chỉ số đánh giá thống nhất [3]. Ngoài ra, Miller và cộng sự cho thấy mô hình dự báo vẫn có thể tạo ra sai số lớn do thiếu thông tin về vận hành thực tế, lịch sử sử dụng, thời tiết hoặc đặc điểm chi tiết của tòa nhà [4]. Vì vậy, không thể mặc định mọi sai số dự báo lớn đều là sự cố thực tế. Ngưỡng cảnh báo cần được hiệu chỉnh trên dữ liệu kiểm chứng và kết quả phải được đánh giá bằng các kịch bản bất thường có kiểm soát.

Một vấn đề khác là phần lớn các nghiên cứu dự báo được thực hiện theo phương thức xử lý theo lô. Toàn bộ dữ liệu được thu thập trước, sau đó mới được đưa vào mô hình để phân tích. Cách làm này phù hợp với báo cáo lịch sử nhưng chưa đáp ứng tốt yêu cầu cảnh báo sớm khi dữ liệu cảm biến liên tục phát sinh. Wang và cộng sự đã đề xuất kiến trúc xử lý luồng cho dữ liệu Internet vạn vật công nghiệp, trong đó mô hình được huấn luyện bằng dữ liệu lịch sử và được sử dụng để phát hiện bất thường trên luồng dữ liệu trực tuyến [5]. Kết quả nghiên cứu cho thấy việc kết hợp huấn luyện theo lô, xử lý luồng và cửa sổ trượt có thể hỗ trợ phát hiện bất thường với độ trễ thấp, đồng thời nâng cao khả năng mở rộng của hệ thống.

Từ các nghiên cứu trên có thể nhận thấy rằng dữ liệu, mô hình dự báo, cơ chế phát hiện bất thường và kiến trúc xử lý thời gian thực là các thành phần có quan hệ chặt chẽ. Tuy nhiên, một mô hình học máy hoạt động độc lập chưa tạo thành một hệ thống Big Data hoàn chỉnh. Để đưa kết quả phân tích vào ứng dụng thực tế, cần xây dựng một chuỗi xử lý bao gồm tiếp nhận dữ liệu, xử lý luồng, lưu trữ lịch sử, huấn luyện mô hình, tạo cảnh báo và trực quan hóa kết quả. Vì vậy, đề tài này tập trung xây dựng một pipeline

Big Data đầu cuối trên dữ liệu BDG2, thay vì chỉ tập trung đề xuất một mô hình học máy mới.

2. Đặt vấn đề

Trong một khuôn viên có nhiều tòa nhà, mỗi công tơ có thể liên tục tạo ra dữ liệu về mức tiêu thụ năng lượng. Khi số lượng công tơ tăng lên, người quản lý khó có thể theo dõi thủ công từng chuỗi dữ liệu để nhận biết tòa nhà nào đang sử dụng năng lượng không bình thường. Nếu dữ liệu chỉ được tổng hợp vào cuối ngày hoặc cuối tháng, các trường hợp như thiết bị hoạt động ngoài giờ, mức tiêu thụ tăng đột biến, cảm biến bị lỗi hoặc dữ liệu bị gián đoạn có thể không được phát hiện kịp thời. Điều này làm giảm hiệu quả vận hành và có thể gây lãng phí năng lượng trong thời gian dài.

Bài toán đặt ra là xây dựng một hệ thống có khả năng tiếp nhận dữ liệu từ nhiều công tơ, xử lý dữ liệu ngay khi phát sinh, lưu trữ lịch sử một cách có tổ chức và xác định những số đo có mức sai lệch đáng kể so với trạng thái vận hành thông thường. Hệ thống không chỉ cần đưa ra kết quả dự báo mà còn phải biến kết quả đó thành thông tin có thể sử dụng, chẳng hạn danh sách tòa nhà cần kiểm tra, thời điểm xuất hiện bất thường và mức độ chênh lệch giữa tiêu thụ thực tế với mức dự kiến.

Dữ liệu sau khi xử lý được lưu trữ bằng Delta Lake theo ba tầng. Tầng Bronze lưu dữ liệu nguyên bản nhận từ Kafka; tầng Silver lưu dữ liệu đã được làm sạch và bổ sung thông tin; tầng Gold lưu các đặc trưng, giá trị dự đoán và kết quả cảnh báo. Cách tổ chức này giúp bảo toàn dữ liệu gốc, hỗ trợ xử lý lại khi cần thiết và phân tách rõ dữ liệu phục vụ lưu trữ với dữ liệu phục vụ phân tích.

Ở giai đoạn phân tích, đề tài dự kiến sử dụng Spark MLlib để xây dựng mô hình ước lượng mức tiêu thụ điện bình thường dựa trên thời gian, điều kiện thời tiết, đặc điểm tòa nhà và các giá trị tiêu thụ trước đó. Độ lệch tuyệt đối giữa giá trị thực tế và giá trị dự đoán được xác định theo biểu thức:

$$e_t = |y_t - \hat{y}_t|$$

trong đó $y_t$là mức tiêu thụ thực tế tại thời điểm t, $\hat{y}_t$là mức tiêu thụ do mô hình dự đoán và $e_t$là sai số dự báo. Khi e_tvượt quá ngưỡng được xác định trên tập dữ liệu kiểm chứng, bản ghi sẽ được đánh dấu là trường hợp bất thường cần xem xét.

Do BDG2 không cung cấp nhãn bất thường trực tiếp, đề tài không giả định rằng mọi số đo có sai số lớn đều là sự cố thực tế. Nhóm sẽ tạo một tập kiểm thử bằng cách

chèn có kiểm soát một số dạng bất thường như mức tiêu thụ tăng đột biến, giá trị giảm về 0, cảm biến giữ nguyên một giá trị trong thời gian dài, bản ghi bị trùng hoặc dữ liệu đến trễ. Các trường hợp này chỉ được sử dụng để đánh giá khả năng phát hiện của hệ thống và sẽ được phân biệt rõ với dữ liệu gốc. Kết quả của hệ thống được đánh giá trên hai phương diện. Về mô hình, các chỉ số MAE và RMSE được sử dụng để đánh giá sai số dự báo; Precision, Recall và F1-score được sử dụng để đánh giá khả năng phát hiện các bất thường đã được chèn. Về hệ thống, đề tài xem xét số bản ghi xử lý trong một đơn vị thời gian, độ trễ từ khi Kafka nhận dữ liệu đến khi cảnh báo được tạo, khả năng khôi phục từ checkpoint và khả năng hạn chế ghi trùng khi chương trình được khởi động lại. Trong phạm vi triển khai ban đầu, đề tài tập trung vào dữ liệu điện và lựa chọn một số lượng tòa nhà phù hợp với tài nguyên máy tính của nhóm. Pipeline cốt lõi gồm Python Producer, Kafka, Spark Structured Streaming, Delta Lake và Spark MLlib. MongoDB, Airflow và giao diện trực quan hóa sẽ được bổ sung sau khi luồng xử lý chính hoạt động ổn định. Cách giới hạn phạm vi này giúp bảo đảm tính khả thi nhưng vẫn thể hiện đầy đủ quá trình tiếp nhận, xử lý, lưu trữ và phân tích dữ liệu lớn.

Từ những vấn đề nêu trên, đề tài được thực hiện với tên gọi: "Xây dựng pipeline Big Data thời gian thực để giám sát và phát hiện bất thường trong tiêu thụ năng lượng tòa nhà". Kết quả kỳ vọng là một hệ thống đầu cuối có khả năng mô phỏng luồng dữ liệu từ nhiều công tơ, xử lý và lưu trữ dữ liệu có cấu trúc, dự đoán mức tiêu thụ thông thường, đồng thời tạo cảnh báo khi phát hiện độ lệch đáng kể.

3. Mục tiêu của đề tài

Mục tiêu tổng quát: Xây dựng thành công hệ thống thu thập, xử lý và phân tích dữ liệu cảm biến điện năng theo thời gian thực, có khả năng dự báo nhu cầu tiêu thụ và tự động phát hiện, cảnh báo các bất thường nhằm tối ưu hóa việc quản lý năng lượng và đảm bảo an toàn cho mạng lưới điện trong các tòa nhà thông minh.

Mục tiêu cụ thể:

- Truyền tải thời gian thực (Kafka & Spark Streaming) và xử lý dữ liệu tốc độ cao tiếp nhận và xử lý song song luồng dữ liệu từ hàng nghìn công tơ điện tử với độ trễ thấp (end - to end latency)
- Lưu trữ kết hợp giữa xử lý lịch sử khối lượng lớn và truy xuất trạng thái nhanh (NoSQL) để đáp ứng đa dạng yêu cầu từ báo cáo đến cảnh báo.

- Phân tích và dự báo: Phát triển và triển khai tự động hóa các mô hình học máy để học hành vi tiêu thụ điện thông thường của tòa nhà, từ đó dự báo nhu cầu tương lai và đánh giá các điểm dữ liệu bất thường (anomaly detection)

4. Nội dung nghiên cứu

. a) Lý thuyết

- Nghiên cứu đặc điểm của time-series data từ các cảm biến IoT và công tơ điện
- Khảo sát kiến trúc xử lý dữ liệu lớn theo mô hình Kappa và các công nghệ cốt lõi: Apache Kafka, Apache Spark

b) Luồng xử lý dữ liệu thời gian thực

- Apache Kafka: Thiết lập các Kafka Brokers và Topics để làm message broker, tiếp nhận luồng dữ liệu tiêu thụ điện liên tục phát ra từ công tơ điện
- Spark Streaming: Viết các job xử lý bằng Spark Streaming để tiêu thụ dữ liệu từ Kafka. Thực hiện làm sạch dữ liệu, chuyển đổi định dạng và tính toán các chỉ số điện năng theo thời gian thực.
- Lưu trữ dùng Delta Lake để lưu trữ lượng lớn dữ liệu lịch sử từ đó phân tích và huấn luận mô hình học máy, lưu trữ trạng thái cảnh báo bằng MongoDB

c) Mô hình Machine Learning dự báo và cảnh báo

- Spark MLlib: Trích xuất dữ liệu lịch sử từ Delta Lake để đưa vào thư viện Spark MLlib. Xây dựng các mô hình dự báo chuỗi thời gian (Random Forest) về mức tiêu thụ bình thường của tòa nhà theo từng khung giờ/ngày.
- Anomaly Detection:Dữ liệu thời gian thực Spark Streaming với mức dự báo từ MLlib để phát hiện độ lệch lớn, từ đó gán nhãn dữ liệu là "bất thường" (ví dụ: rò rỉ điện, thiết bị hỏng, nguy cơ quá tải).

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về dữ liệu tiêu thụ điện theo chuỗi thời gian

Bộ dữ liệu được xây dựng nhằm phục vụ bài toán mô hình hóa mức tiêu thụ năng lượng của các tòa nhà dựa trên dữ liệu sử dụng trong quá khứ và điều kiện thời tiết quan sát được. Mục tiêu là xây dựng mô hình phản thực (*counterfactual model*) để ước lượng mức năng lượng mà một tòa nhà có thể đã tiêu thụ nếu chưa thực hiện các biện pháp cải thiện hiệu quả năng lượng. Dữ liệu bao gồm khoảng ba năm số đo công tơ theo giờ của hơn một nghìn tòa nhà tại nhiều địa điểm khác nhau trên thế giới. Bốn loại năng lượng được xem xét gồm điện năng, nước lạnh, hơi nước và nước nóng. Mỗi loại được xác định thông qua biến meter, trong đó giá trị 0, 1, 2 và 3 lần lượt tương ứng với electricity, chilled water, steam và hot water. Ngoài dữ liệu tiêu thụ, bộ dữ liệu còn cung cấp thông tin đặc trưng của từng tòa nhà và dữ liệu thời tiết tại khu vực tương ứng. Đây là cơ sở để phân tích mối liên hệ giữa đặc điểm công trình, điều kiện môi trường và mức tiêu thụ năng lượng theo thời gian.

2.1.1 Dữ liệu cảm biến và công tơ năng lượng

Dữ liệu tiêu thụ năng lượng chính được lưu trong tệp train.csv. Mỗi bản ghi tương ứng với một giá trị đo tại một tòa nhà, một loại công tơ và một thời điểm cụ thể. Các thuộc tính chính gồm:

Thuộc tính	**Ý nghĩa**
building_id	Mã định danh của tòa nhà
meter	Loại công tơ năng lượng
timestamp	Thời điểm ghi nhận dữ liệu
meter_reading	Giá trị tiêu thụ năng lượng

Biến meter được mã hóa như sau:

Giá trị	**Loại năng lượng**
0	Electricity
1	Chilled water
2	Steam
3	Hot water

Không phải mọi tòa nhà đều có đầy đủ bốn loại công tơ. Do đó, số lượng chuỗi dữ liệu của từng tòa nhà có thể khác nhau.

Biến meter_reading là biến mục tiêu, thể hiện lượng năng lượng tiêu thụ theo kWh hoặc đơn vị tương đương. Tuy nhiên, dữ liệu điện tại site_id = 0 được ghi nhận theo đơn vị kBTU, đây là điểm cần được lưu ý trong quá trình tiền xử lý và phân tích.

Thông tin bổ sung của từng tòa nhà được lưu trong building_meta.csv, bao gồm site_id, primary_use, square_feet, year_built và floor_count. Các thuộc tính này mô tả vị trí, chức năng sử dụng, diện tích, năm xây dựng và số tầng của tòa nhà.

2.1.2 Dữ liệu chuỗi thời gian

Các giá trị tiêu thụ năng lượng được ghi nhận theo từng giờ, do đó dữ liệu có cấu trúc chuỗi thời gian. Với một tòa nhà bb và loại công tơ mm, chuỗi tiêu thụ năng lượng có thể biểu diễn:

$$X = \{x_1, x_2, \dots, x_T\}$$

trong đó x_t là giá trị meter_reading tại thời điểm t.

Một chuỗi dữ liệu được xác định chủ yếu bởi ba thành phần:

(building_id, meter, timestamp)

Việc sử dụng timestamp cho phép khai thác các đặc trưng thời gian như giờ trong ngày, ngày trong tuần, tháng hoặc mùa để phân tích quy luật sử dụng năng lượng.

Bộ dữ liệu còn có weather_train.csv và weather_test.csv, cung cấp dữ liệu khí tượng của trạm gần từng khu vực tòa nhà nhất. Các biến gồm:

- air_temperature: nhiệt độ không khí;
- cloud_coverage: độ che phủ mây;
- dew_temperature: nhiệt độ điểm sương;
- precip_depth_1_hr: lượng mưa trong một giờ;
- sea_level_pressure: áp suất mực nước biển;
- wind_direction: hướng gió;
- wind_speed: tốc độ gió.

Dữ liệu thời tiết được liên kết với dữ liệu tòa nhà thông qua site_id và thời gian đo, từ đó hỗ trợ phân tích ảnh hưởng của điều kiện môi trường đến nhu cầu sử dụng năng lượng.

2.1.3 Đặc điểm dữ liệu

- D**ữ liệu đa chuỗi thời gian**. Mỗi tổ hợp giữa building_id và meter có thể được xem như một chuỗi tiêu thụ năng lượng độc lập.

- Dữ liệu có **độ phân giải theo giờ** và kéo dài trong khoảng ba năm. Vì vậy, số lượng quan sát của mỗi tòa nhà có thể lớn, đặc biệt khi tòa nhà có loại công tơ.
- Dữ liệu có **tính không đồng nhất giữa các tòa nhà**. Các tòa nhà khác nhau về chức năng sử dụng, diện tích, tuổi công trình và số tầng, do đó mức tiêu thụ năng lượng cũng có thể khác biệt đáng kể.
- Dữ liệu tiêu thụ có thể chịu ảnh hưởng của **điều kiện thời tiết**. Các yếu tố như nhiệt độ không khí, độ ẩm gián tiếp thông qua nhiệt độ điểm sương, lượng mưa hoặc tốc độ gió có thể liên quan đến nhu cầu sưởi ấm hoặc làm mát của tòa nhà.
- Các loại năng lượng có đặc điểm khác nhau và không phải tòa nhà nào cũng có đủ bốn loại công tơ. Vì vậy, việc phân tích cần xem xét riêng từng loại meter thay vì coi toàn bộ meter_reading là một chuỗi đồng nhất.

2.1.4 Các vấn đề chất lượng dữ liệu cảm biến

Do đây là dữ liệu đo thực tế, chất lượng dữ liệu không hoàn toàn lý tưởng. Mô tả dataset cho biết biến meter_reading chứa **sai số đo lường**, do đó tồn tại một mức sai số nền ngay cả khi mô hình dự báo được xây dựng tốt. Bên cạnh đó, dữ liệu có **các khoảng trống trong chuỗi công tơ** ở cả tập huấn luyện và tập kiểm tra. Điều này làm cho một số chuỗi không liên tục hoàn toàn theo từng giờ. Một vấn đề khác là **không đồng nhất về đơn vị đo**. Cụ thể, dữ liệu điện của site_id = 0 được ghi nhận theo kBTU, trong khi phần lớn dữ liệu còn lại được mô tả theo kWh hoặc đơn vị năng lượng tương đương. Nếu không kiểm tra vấn đề này, quá trình thống kê hoặc so sánh giữa các site có thể bị sai lệch. Ngoài ra, dữ liệu metadata và dữ liệu thời tiết có thể không đầy đủ ở một số bản ghi, vì vậy cần kiểm tra dữ liệu thiếu trước khi xây dựng mô hình.

Do đó, quá trình tiền xử lý nên tập trung vào bốn nhóm vấn đề chính: **missing values, gaps in time series, measurement error và unit inconsistency**. Đặc biệt, không nên tự động xem mọi giá trị bằng 0 hoặc mọi giá trị lớn là lỗi, vì mô tả dataset không khẳng định điều đó. Việc xác định dữ liệu bất thường cần dựa trên phân tích thực tế của từng building_id, từng meter và từng khoảng thời gian.

2.2. Xử lý dữ liệu lớn và dữ liệu thời gian thực

2.2.1. Xử lý theo lô và xử lý luồng

2.2.3. Batch Processing và Stream Processing

2.2.5 Kiến trúc Kappa

2.4. Lưu trữ dữ liệu

2.4.1. Delta Lake và kiến trúc Bronze – Silver – Gold

Đối với dữ liệu tiêu thụ năng lượng theo chuỗi thời gian, hệ thống lưu trữ cần đáp ứng khả năng mở rộng, hỗ trợ dữ liệu lịch sử lớn và cho phép xử lý hiệu quả bằng các công cụ phân tích như Spark. Trong kiến trúc Lakehouse, dữ liệu thường được lưu trên nền tảng lưu trữ mở như HDFS hoặc object storage, trong khi lớp quản lý giao dịch bổ sung các cơ chế như ACID transaction, versioning và quản lý schema.

Đối với dữ liệu tiêu thụ năng lượng theo chuỗi thời gian, hệ thống lưu trữ cần có khả năng xử lý khối lượng dữ liệu lớn, duy trì dữ liệu lịch sử và hỗ trợ hiệu quả cho các tác vụ xử lý phân tán bằng Apache Spark. Trong hệ thống được xây dựng, dữ liệu được tổ chức theo kiến trúc nhiều lớp Bronze – Silver – Gold, trong đó mỗi lớp tương ứng với một mức độ xử lý và hoàn thiện khác nhau của dữ liệu.

Các tập dữ liệu trung gian được lưu dưới định dạng Apache Parquet. Đây là định dạng lưu trữ dạng cột, phù hợp với các bài toán phân tích dữ liệu lớn do chỉ cần đọc những cột cần thiết thay vì toàn bộ bản ghi. Parquet đồng thời hỗ trợ schema và khả năng nén dữ liệu, nhờ đó giảm dung lượng lưu trữ và lượng dữ liệu phải đọc trong quá trình xử lý bằng Spark.

- Bronze Layer là lớp lưu trữ đầu tiên sau dữ liệu thô. Dữ liệu từ các tập train.csv, building_metadata.csv và weather_train.csv được Spark đọc với schema xác định trước, sau đó chuyển đổi một số kiểu dữ liệu cơ bản trước khi lưu thành Parquet. Hệ thống đồng thời bổ sung trường _ingested_at để ghi nhận thời điểm dữ liệu được đưa vào pipeline và _source_file để xác định nguồn của mỗi bản ghi. Dữ liệu Bronze được lưu riêng thành các tập train, building_metadata và weather_train trong thư mục data/bronze.
- Silver Layer chứa dữ liệu đã được làm sạch, chuẩn hóa và tích hợp từ nhiều nguồn. Tại lớp này, hệ thống loại bỏ các bản ghi thiếu các trường quan trọng như building_id, meter, timestamp và meter_reading, đồng thời loại bỏ các bản ghi

trùng lặp dựa trên khóa nghiệp vụ. Dữ liệu tiêu thụ năng lượng sau đó được kết hợp với thông tin tòa nhà theo building_id và dữ liệu thời tiết theo cặp khóa site_id và timestamp. Ngoài ra, một số đặc trưng thời gian như giờ trong ngày, ngày trong tuần, tháng và trạng thái cuối tuần được tạo thêm để phục vụ các bước phân tích tiếp theo. Kết quả được lưu tại data/silver/ashrae_joined.

- Gold Layer là lớp dữ liệu đã được chuẩn bị cho các tác vụ phân tích và xây dựng mô hình học máy. Từ dữ liệu Silver, hệ thống tiếp tục xử lý các giá trị khuyết thiếu của metadata tòa nhà và dữ liệu thời tiết. Với dữ liệu thời tiết, phương pháp forward fill và backward fill theo từng site_id được sử dụng trước, sau đó giá trị trung vị được dùng làm phương án dự phòng cho các giá trị còn thiếu.

Bên cạnh làm sạch dữ liệu, lớp Gold còn thực hiện feature engineering, bao gồm biến đổi logarithm cho meter_reading và square_feet, tính tuổi tòa nhà, tạo các đặc trưng chu kỳ cho giờ và tháng bằng hàm sine/cosine, cũng như tính chênh lệch giữa nhiệt độ không khí và nhiệt độ điểm sương. Dữ liệu Gold cuối cùng được lưu dưới dạng Parquet và phân vùng theo trường meter, giúp hạn chế lượng dữ liệu cần quét khi thực hiện các truy vấn theo loại đồng hồ đo. Như vậy, kiến trúc Bronze – Silver – Gold giúp pipeline phân tách rõ dữ liệu theo từng giai đoạn xử lý: Bronze lưu dữ liệu gần với nguồn ban đầu, Silver cung cấp dữ liệu đã làm sạch và tích hợp, còn Gold cung cấp dữ liệu đã được hoàn thiện và biến đổi đặc trưng phục vụ phân tích hoặc học máy. Cách tổ chức này giúp quá trình xử lý dữ liệu dễ kiểm soát, tái sử dụng và mở rộng hơn khi khối lượng dữ liệu tăng lên.

2.5 Học máy và mô hình ước lượng tiêu thụ điện

2.5.1 Xác định bài toán hồi quy

Với mỗi tòa nhà tại một giờ quan sát, bài toán cần ước lượng điện năng tiêu thụ kỳ vọng từ thông tin tòa nhà, đặc trưng lịch và thời tiết. Giá trị meter_reading được dùng làm biến mục tiêu. Trong bản điện RAW của BDG2, đơn vị năng lượng là kWh và độ phân giải thời gian là một giờ [1]. Vì vậy, kết quả được trình bày theo điện năng trong giờ đo; không gán trực tiếp thành công suất tức thời kW.

Vector đầu vào gồm mã tòa nhà, mã site, công năng, diện tích, số tầng, năm xây dựng và thời tiết cùng giờ. Các đặc trưng lịch biểu diễn chu kỳ giờ trong ngày, ngày trong

tuần và tháng trong năm. meter_reading của chính bản ghi đang đánh giá không được đưa vào vector đầu vào, nhằm tránh việc dùng đáp án làm đặc trưng.

Thiết kế hiện tại sử dụng thời tiết quan sát cùng giờ để tính mức tiêu thụ kỳ vọng. Cách thiết kế này phù hợp với việc so sánh phép đo vừa nhận với một mức tham chiếu có điều kiện. Muốn đánh giá dự báo trước một giờ hoặc một ngày, cần xác định thời hạn dự báo, dùng dữ liệu sẵn có trước thời điểm dự báo và xây dựng lại phép đánh giá.

2.5.2 Nguyên lý Random Forest Regression

Random Forest Regression kết hợp nhiều cây quyết định hồi quy. Các cây được tạo với lấy mẫu dữ liệu và lựa chọn tập con đặc trưng tại các bước chia nút; đầu ra của rừng là trung bình đầu ra các cây [2]. Với B cây và vector đầu vào x, giá trị ước lượng được biểu diễn như sau:

$$\hat{y}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^{B} T_b(\mathbf{x})$$

Trong đó, B là số cây, b là chỉ số cây và T là hàm dự đoán của từng cây. Mỗi cây biểu diễn một tập các điều kiện phân nhánh trên đặc trưng, còn phép lấy trung bình làm giảm sự phụ thuộc vào một cây riêng lẻ. Với dữ liệu điện, cấu trúc cây cho phép biểu diễn các chế độ khác nhau theo thời tiết, quy mô và lịch vận hành.

Việc tăng số cây hoặc độ sâu không tự bảo đảm chất lượng tốt hơn trên dữ liệu chưa gặp. Cây sâu có thể mô tả chi tiết dữ liệu huấn luyện nhưng vẫn sai khi chế độ vận hành thay đổi. Do đó, cấu hình cần được đánh giá theo thời gian và đối chiếu với một phương pháp đơn giản trên cùng tập test. Trong pilot này, RF có vai trò tạo mức tham chiếu; kết luận về hiệu quả được dựa trên số đo thay vì chỉ dựa trên đặc tính của thuật toán.

2.5.3 Triển khai bằng Spark MLlib

Thành phần học máy sử dụng API DataFrame của Spark MLlib, với RandomForestRegressor nằm trong một Pipeline. Pipeline đã huấn luyện được lưu để dùng lại trong bước scoring [2]. Thiết kế này giúp áp dụng thống nhất phép mã hóa và cấu hình mô hình giữa đánh giá lịch sử và xử lý bản ghi mới.

Phần tiền xử lý dùng StringIndexer để tạo chỉ số cho biến phân loại, OneHotEncoder để tạo biểu diễn phân loại và VectorAssembler để ghép các đầu vào [3]. Bộ triển khai bổ sung xử lý thiếu bằng median train, cờ thiếu và đặc trưng lịch. Các tham số này được giữ cùng model, nhằm hạn chế sai khác khi tích hợp vào micro-batch.

2.6 Sai số dự báo, phát hiện bất thường và chỉ số đánh giá

2.6.1 Sai số dự báo và mức độ lệch

Gọi y_i là giá trị điện năng đo được và $\hat{y}_i$ là mức tiêu thụ kỳ vọng của bản ghi i. Residual và độ lớn residual được xác định bởi:

$$r_i = y_i - \hat{y}_i, \qquad a_i = |r_i|$$

Residual dương thể hiện phép đo lớn hơn mức tham chiếu; residual âm thể hiện phép đo thấp hơn mức tham chiếu. Độ lớn residual cho biết mức chênh lệch cần xem xét. Một chênh lệch lớn có thể do chế độ vận hành bất thường, thay đổi công năng, dữ liệu thời tiết thiếu hoặc mô hình biểu diễn chưa tốt; nó chưa đủ để khẳng định đã xảy ra sự cố điện.

2.6.2 Chỉ số đánh giá hồi quy

MAE là trung bình độ lớn sai số. RMSE là căn bậc hai trung bình bình phương sai số và nhạy hơn với các sai số lớn. Cả hai cùng đơn vị kWh với biến mục tiêu của pilot:

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

Hệ số xác định R^2 so sánh tổng bình phương sai số với độ phân tán của giá trị mục tiêu trên tập đánh giá:

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}$$

Trong đó, $\bar{y}$ là trung bình của y trên tập đang đánh giá. R^2 có thể âm khi mô hình có sai số lớn hơn việc dùng trung bình của tập đó. Khi toàn bộ giá trị mục tiêu không đổi, mẫu số bằng 0 nên R^2 không được xác định. R^2 không được diễn giải thành phần trăm dự đoán đúng; với nhiều tòa nhà có mức tiêu thụ khác nhau, cần đọc thêm chỉ số từng tòa nhà.

2.6.3 Chọn ngưỡng bất thường

Detector sử dụng độ lớn residual so với ngưỡng chọn từ validation. Tại tòa nhà b có đủ mẫu, ngưỡng là phân vị 99% của độ lớn residual validation, có giới hạn nhỏ để tránh ngưỡng bằng 0:

$$\delta_b = \max\left\{Q_{0.99}\left(|r|_{\text{validation},b}\right),\ 10^{-6}\right\}$$

Nếu tòa nhà có dưới 48 mẫu validation, detector dùng ngưỡng chung. Cách chọn trên validation giữ tập test cho đánh giá cuối cùng. Do dữ liệu lịch sử chưa được xác nhận hoàn toàn bình thường và phân phối có thể thay đổi, quy tắc phân vị 99% không bảo đảm tỷ lệ cảnh báo sai thực tế luôn bằng 1%.

2.6.4 Precision, Recall và F1 score

Khi có nhãn, TP là mẫu dương được cảnh báo, FN là mẫu dương không được cảnh báo, FP là mẫu âm được cảnh báo và TN là mẫu âm không được cảnh báo. Các chỉ số được tính như sau:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2TP}{2TP + FP + FN}$$

Trong thực nghiệm chèn thay đổi, nhãn dương chỉ phép đo được sửa có chủ đích; nhãn âm chỉ phép đo gốc giữ nguyên. Theo đó, Precision, Recall và F1 đánh giá khả năng nhận biết các thay đổi đã tạo. Các phép đo gốc chưa có nhãn xác nhận sự cố, nên ma trận nhầm lẫn của thực nghiệm này không được dùng để công bố độ chính xác phát hiện sự cố ngoài thực tế.

CHƯƠNG 3: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

3.1. Phân tích yêu cầu và kiến trúc tổng thể hệ thống

Hệ thống được xây dựng nhằm xử lý và phân tích dữ liệu tiêu thụ năng lượng của các tòa nhà thông minh trên tập dữ liệu ASHRAE. Dữ liệu đầu vào bao gồm thông tin tiêu thụ năng lượng, thông tin đặc trưng của tòa nhà và dữ liệu thời tiết theo thời gian. Do dữ liệu có kích thước lớn, nhiều nguồn và mang đặc trưng chuỗi thời gian, hệ thống cần tổ chức pipeline theo từng tầng xử lý riêng biệt nhằm đảm bảo khả năng làm sạch, tích hợp, biến đổi và tái sử dụng dữ liệu.

Kiến trúc hệ thống kết hợp Apache Spark cho xử lý dữ liệu phân tán, Apache Parquet cho lưu trữ dữ liệu trung gian theo kiến trúc Bronze – Silver – Gold và Apache Kafka để mô phỏng quá trình truyền dữ liệu tiêu thụ năng lượng theo thời gian thực

3.1.1. Yêu cầu chức năng và phi chức năng

Hệ thống cần đáp ứng các chức năng chính sau:

- Đọc và tiếp nhận dữ liệu từ nhiều nguồn: hệ thống sử dụng ba tập dữ liệu chính gồm dữ liệu tiêu thụ năng lượng train.csv, thông tin tòa nhà building_metadata.csv và dữ liệu thời tiết weather_train.csv. Spark được sử dụng để đọc các tập dữ liệu này và áp dụng schema tương ứng cho từng nguồn.
- Chuyển đổi dữ liệu thô sang lớp Bronze: dữ liệu đầu vào được chuẩn hóa kiểu dữ liệu, đặc biệt là trường thời gian, đồng thời bổ sung thông tin về thời điểm ingest và nguồn dữ liệu thông qua các trường _ingested_at và _source_file.
- Làm sạch và tích hợp dữ liệu: tại lớp Silver, hệ thống loại bỏ các bản ghi thiếu các trường quan trọng và loại bỏ dữ liệu trùng lặp. Dữ liệu tiêu thụ năng lượng được liên kết với metadata tòa nhà theo building_id, sau đó kết hợp với thông tin thời tiết theo site_id và timestamp.
- Bổ sung đặc trưng thời gian: hệ thống tạo thêm các thuộc tính như giờ trong ngày, ngày trong tuần, ngày trong tháng, tháng và trạng thái cuối tuần để hỗ trợ các bước phân tích dữ liệu tiếp theo.
- Xử lý dữ liệu thiếu: tại lớp Gold, các giá trị thiếu của thông tin tòa nhà được xử lý bằng giá trị trung vị. Đối với dữ liệu thời tiết, hệ thống áp dụng forward fill, backward fill theo từng site_id, kết hợp với median fallback trong trường hợp vẫn còn giá trị thiếu.

- Xây dựng đặc trưng phục vụ phân tích và học máy: hệ thống thực hiện log-transform cho dữ liệu tiêu thụ năng lượng và diện tích tòa nhà, tính tuổi tòa nhà, tạo đặc trưng chu kỳ của giờ và tháng, đồng thời xây dựng biến chênh lệch nhiệt độ từ dữ liệu thời tiết.
- Lưu trữ dữ liệu theo nhiều tầng: kết quả của từng giai đoạn được lưu dưới định dạng Parquet tương ứng với các lớp Bronze, Silver và Gold. Dữ liệu Gold được phân vùng theo loại đồng hồ meter nhằm hỗ trợ truy vấn dữ liệu hiệu quả hơn.
- Mô phỏng dữ liệu thời gian thực bằng Kafka: dữ liệu đã được xử lý tại lớp Silver có thể được đọc theo từng bản ghi và gửi vào Kafka topic energy_raw. Producer cho phép điều chỉnh tốc độ gửi, kích thước batch và số lượng bản ghi để phục vụ kiểm thử hiệu năng.
- Tiếp nhận và kiểm tra dữ liệu Kafka: Kafka Consumer đọc các message từ topic energy_raw, giải mã dữ liệu JSON và hiển thị các thông tin như tòa nhà, timestamp, mức tiêu thụ năng lượng và nhiệt độ.

Ngoài các chức năng xử lý dữ liệu, hệ thống cần đáp ứng một số yêu cầu phi chức năng sau:

- Khả năng mở rộng: hệ thống sử dụng Apache Spark nhằm hỗ trợ xử lý lượng dữ liệu lớn và có thể mở rộng từ môi trường cục bộ sang môi trường cluster khi cần thiết.
- Hiệu năng xử lý: dữ liệu được lưu dưới định dạng Parquet dạng cột nhằm giảm lượng dữ liệu cần đọc khi chỉ truy vấn một số thuộc tính nhất định. Trong Spark, số lượng partition và dung lượng bộ nhớ driver có thể được cấu hình để tối ưu quá trình xử lý. Trong pipeline Silver, metadata tòa nhà còn được thực hiện broadcast join nhằm giảm chi phí shuffle khi kết hợp với bảng dữ liệu tiêu thụ năng lượng lớn.
- Đảm bảo chất lượng dữ liệu: tại các giai đoạn xử lý, hệ thống thực hiện kiểm tra số lượng bản ghi, giá trị null, dữ liệu trùng lặp và các trường dữ liệu quan trọng trước khi lưu sang tầng tiếp theo.
- Khả năng tái sử dụng dữ liệu: việc phân chia thành Bronze, Silver và Gold giúp các bước xử lý được tách biệt. Dữ liệu tại mỗi lớp có thể được sử dụng lại mà không cần thực hiện toàn bộ pipeline từ đầu.

- Khả năng theo dõi nguồn dữ liệu: các trường _ingested_at và _source_file được bổ sung ở tầng Bronze nhằm hỗ trợ việc truy vết thời điểm ingest và nguồn gốc của dữ liệu.
- Khả năng xử lý dữ liệu dòng: Kafka được sử dụng như một lớp truyền tải dữ liệu, hỗ trợ producer và consumer hoạt động độc lập. Producer sử dụng building_id làm message key, qua đó các bản ghi liên quan đến cùng một tòa nhà có thể được định tuyến nhất quán theo partition của Kafka.

3.1.2. Kiến trúc tổng thể

Kiến trúc tổng thể của hệ thống được chia thành các lớp chính gồm Data Source, Data Processing, Data Storage và Streaming Layer.

- Ở lớp Data Source, dữ liệu ban đầu được lấy từ bộ dữ liệu ASHRAE, bao gồm ba nhóm dữ liệu chính: thông tin tiêu thụ năng lượng của từng tòa nhà, metadata của tòa nhà và thông tin thời tiết tương ứng theo từng site và thời điểm.
- Tiếp theo là lớp Data Processing, sử dụng Apache Spark để thực hiện các bước ETL. Pipeline xử lý dữ liệu được tổ chức theo kiến trúc Bronze – Silver – Gold:
- Bronze Layer tiếp nhận dữ liệu gần với trạng thái ban đầu, chuẩn hóa kiểu dữ liệu và bổ sung metadata ingest.
- Silver Layer thực hiện làm sạch dữ liệu, loại bỏ dữ liệu thiếu hoặc trùng lặp, kết hợp dữ liệu tiêu thụ năng lượng với thông tin tòa nhà và dữ liệu thời tiết, đồng thời bổ sung các đặc trưng thời gian.
- Gold Layer tiếp tục xử lý dữ liệu thiếu và thực hiện feature engineering để tạo ra tập dữ liệu hoàn chỉnh, phù hợp cho các bài toán phân tích và xây dựng mô hình học máy.
- Ở lớp Data Storage, dữ liệu của từng tầng được lưu dưới định dạng Apache Parquet. Cấu trúc thư mục của hệ thống được tổ chức tương ứng với từng lớp:
- data/raw → data/bronze → data/silver → data/gold

Trong đó, lớp Silver được lưu tại data/silver/ashrae_joined, còn lớp Gold được lưu tại data/gold/ashrae_ml_ready. Ngoài pipeline xử lý batch, hệ thống còn có Streaming Layer sử dụng Apache Kafka. Dữ liệu từ lớp Silver được Kafka Producer đọc và chuyển thành các message JSON gửi tới topic energy_raw. Kafka Consumer đăng ký topic này để nhận và kiểm tra dữ liệu theo thời gian thực.

Có thể biểu diễn kiến trúc tổng thể của hệ thống như sau:

ASHRAE Dataset

↓

Raw Data

train.csv + building_metadata.csv + weather_train.csv

↓

Apache Spark

↓

Bronze Layer

Chuẩn hóa schema → chuẩn hóa timestamp → thêm metadata ingest

↓

Silver Layer

Làm sạch → loại bỏ duplicate → join dữ liệu → tạo đặc trưng thời gian

↓

Gold Layer

Xử lý missing value → feature engineering → dữ liệu ML-ready

↓

Parquet Storage

Song song với đó:

Silver Layer → Kafka Producer → Kafka Topic energy_raw → Kafka Consumer

Kiến trúc này giúp tách biệt xử lý dữ liệu batch và truyền tải dữ liệu dạng streaming, đồng thời tạo ra các lớp dữ liệu có mức độ hoàn thiện khác nhau để đáp ứng nhiều mục đích sử dụng.

3.1.3. Luồng dữ liệu trong hệ thống

Luồng dữ liệu của hệ thống bắt đầu từ các tập dữ liệu CSV ban đầu. Spark đọc ba nguồn dữ liệu gồm train.csv, building_metadata.csv và weather_train.csv với schema được xác định trước.

- Ở giai đoạn Raw → Bronze, dữ liệu timestamp trong bảng tiêu thụ năng lượng và bảng thời tiết được chuyển sang kiểu timestamp của Spark. Hệ thống đồng

thời bổ sung thời gian ingest và tên file nguồn, sau đó ghi kết quả xuống thư mục Bronze dưới định dạng Parquet.

- Ở giai đoạn Bronze → Silver, ba tập dữ liệu Parquet được Spark đọc lại. Các bản ghi thiếu khóa hoặc giá trị tiêu thụ năng lượng được loại bỏ, các bản ghi trùng lặp được xử lý, sau đó dữ liệu được join theo hai bước. Đầu tiên, dữ liệu năng lượng được join với metadata tòa nhà bằng building_id. Tiếp theo, kết quả được join với dữ liệu thời tiết bằng site_id và timestamp.
- Sau quá trình join, Spark tạo thêm các đặc trưng thời gian gồm hour, day_of_week, day_of_month, month và is_weekend. Dữ liệu sau bước này được ghi xuống lớp Silver tại data/silver/ashrae_joined.
- Ở giai đoạn Silver → Gold, hệ thống tiếp tục xử lý các giá trị còn thiếu trong dữ liệu metadata và thời tiết. Sau đó, các đặc trưng mới như meter_reading_log1p, square_feet_log1p, building_age, hour_sin, hour_cos, month_sin, month_cos và temperature_diff được xây dựng.
- Sau khi hoàn thành bước feature engineering, hệ thống lựa chọn các trường cần thiết để tạo tập dữ liệu Gold và lưu kết quả tại data/gold/ashrae_ml_ready. Dữ liệu được partition theo trường meter để thuận tiện cho việc truy xuất theo từng loại đồng hồ đo.
- Bên cạnh pipeline batch, dữ liệu tại lớp Silver còn được sử dụng làm nguồn đầu vào cho Kafka Producer. Producer đọc từng bản ghi từ file Parquet, chuyển dữ liệu sang JSON và gửi tới Kafka topic energy_raw. Giá trị building_id được sử dụng làm message key, trong khi nội dung message bao gồm thông tin tiêu thụ năng lượng, thời gian, tòa nhà và điều kiện thời tiết.
- Kafka Consumer sau đó nhận các message từ topic, giải mã nội dung JSON và cung cấp dữ liệu cho bước giám sát hoặc xử lý tiếp theo.

Như vậy, luồng dữ liệu tổng thể có thể tóm tắt như sau:

CSV Raw Data → Spark → Bronze → Spark Cleaning & Join → Silver → Feature Engineering → Gold

đồng thời:

Silver → Kafka Producer → Kafka Broker → Kafka Consumer

Cách tổ chức này cho phép hệ thống vừa xử lý dữ liệu lịch sử theo mô hình batch, vừa mô phỏng quá trình truyền dữ liệu liên tục thông qua Kafka, tạo nền tảng cho việc mở rộng sang các bài toán phân tích và dự đoán tiêu thụ năng lượng theo thời gian thực.

3.2. Chuẩn bị và mô phỏng dữ liệu thời gian thực

3.2.1. Lựa chọn và tiền xử lý dữ liệu BDG2

3.2.2. Xây dựng Python Producer

3.2.3. Định dạng và cơ chế phát dữ liệu

3.3. Xây dựng luồng xử lý dữ liệu thời gian thực

3.3.1. Cấu hình Apache Kafka

3.3.2. Kết nối Kafka với Spark Structured Streaming

3.3.3. Làm sạch và xử lý dữ liệu luồng

3.3.4. Xử lý dữ liệu đến trễ và dữ liệu trùng

3.4. Xây dựng hệ thống lưu trữ dữ liệu

Hệ thống lưu trữ dữ liệu được xây dựng theo mô hình nhiều tầng Bronze - Silver - Gold. Mỗi tầng đảm nhận một mức độ xử lý khác nhau, từ lưu trữ dữ liệu gần với nguồn ban đầu, làm sạch và tích hợp dữ liệu, đến chuẩn bị dữ liệu hoàn chỉnh cho phân tích và xây dựng mô hình học máy. Trong phiên bản hiện tại, dữ liệu tại các tầng được lưu dưới định dạng Apache Parquet và được xử lý bằng Apache Spark.

Cách tổ chức này giúp tách biệt rõ từng giai đoạn trong pipeline dữ liệu. Khi cần thay đổi logic xử lý ở một tầng, dữ liệu của các tầng trước vẫn có thể được tái sử dụng mà không phải thực hiện lại toàn bộ quá trình từ dữ liệu CSV ban đầu. Cấu trúc thư mục chính được tổ chức theo các vùng data/raw, data/bronze, data/silver và data/gold.

3.4.1. Tầng Bronze

Tầng Bronze là tầng lưu trữ đầu tiên sau dữ liệu thô. Dữ liệu đầu vào gồm ba tập tin train.csv, building_metadata.csv và weather_train.csv trong thư mục data/raw. Trong chương trình raw_to_bronze.py, SparkSession được khởi tạo để đọc dữ liệu và thực hiện quá trình chuyển đổi từ Raw sang Bronze.

Để kiểm soát kiểu dữ liệu ngay từ đầu, chương trình định nghĩa schema riêng cho từng tập dữ liệu. Tập train gồm building_id, meter, timestamp và meter_reading; tập building_metadata gồm site_id, building_id, primary_use, square_feet, year_built và floor_count; tập weather_train gồm site_id, timestamp cùng các thuộc tính thời tiết như

air_temperature, cloud_coverage, dew_temperature, precip_depth_1_hr, sea_level_pressure, wind_direction và wind_speed. Việc khai báo schema tường minh giúp Spark đọc dữ liệu ổn định hơn so với việc phụ thuộc hoàn toàn vào cơ chế suy luận kiểu dữ liệu.

Trong quá trình chuyển đổi, trường timestamp của dữ liệu tiêu thụ năng lượng và dữ liệu thời tiết được chuyển từ chuỗi sang kiểu Timestamp với định dạng yyyy-MM-dd HH:mm:ss. Đồng thời, chương trình bổ sung hai trường metadata là _ingested_at và _source_file. Trường _ingested_at ghi nhận thời điểm bản ghi được đưa vào hệ thống, trong khi _source_file lưu tên tập tin nguồn tương ứng. Hai trường này hỗ trợ theo dõi nguồn gốc dữ liệu và kiểm tra quá trình ingest khi cần thiết.

Sau khi chuyển đổi, chương trình thực hiện một số kiểm tra chất lượng cơ bản như đếm số lượng bản ghi của từng bảng, kiểm tra số timestamp bị null và số meter_reading bị null. Các bước kiểm tra này được thực hiện trước khi ghi dữ liệu xuống tầng Bronze nhằm phát hiện sớm các lỗi đầu vào.

Dữ liệu Bronze được ghi dưới định dạng Parquet với chế độ overwrite. Ba vùng lưu trữ đầu ra lần lượt là data/bronze/train, data/bronze/building_metadata và data/bronze/weather_train. Ở giai đoạn này, dữ liệu vẫn giữ gần với cấu trúc nguồn, chưa thực hiện join hoặc feature engineering; mục tiêu chính là chuẩn hóa kiểu dữ liệu, bổ sung metadata ingest và chuyển từ CSV sang định dạng lưu trữ phù hợp hơn cho Spark.

Thành phần	**Đầu vào**	**Xử lý chính**	**Đầu ra**
Train	data/raw/train.csv	Ép schema; chuyển timestamp; thêm _ingested_at và _source_file	data/bronze/train
Building metadata	data/raw/building_metadata.csv	Ép schema; thêm _ingested_at và _source_file	data/bronze/building_metadata

Weather	data/raw/weather_train.csv	Ép schema; chuyển timestamp; thêm _ingested_at và _source_file	data/bronze/weather_train

3.4.2. Tầng Silver

Tầng Silver được xây dựng từ dữ liệu Parquet ở tầng Bronze thông qua chương trình bronze_to_silver.py. Ba tập dữ liệu train, building_metadata và weather_train được đọc lại bằng Spark từ các thư mục tương ứng. Sau đó, hệ thống chỉ giữ lại các cột nghiệp vụ cần thiết để giảm lượng dữ liệu phải xử lý ở các bước tiếp theo.

Đối với dữ liệu tiêu thụ năng lượng, hệ thống loại bỏ các bản ghi có building_id, meter, timestamp hoặc meter_reading bị null, đồng thời loại bỏ bản ghi trùng dựa trên bộ khóa building_id, meter và timestamp. Đối với metadata tòa nhà, các bản ghi thiếu building_id hoặc site_id bị loại bỏ và mỗi building_id chỉ được giữ lại một bản ghi. Dữ liệu thời tiết được kiểm tra site_id và timestamp, sau đó loại bỏ trùng lặp theo cặp khóa site_id và timestamp.

Sau bước làm sạch, dữ liệu được tích hợp theo hai bước. Trước hết, bảng train được join với bảng building_metadata theo khóa building_id. Do bảng metadata tòa nhà có kích thước nhỏ hơn bảng dữ liệu đo, chương trình sử dụng broadcast join để giảm chi phí shuffle. Tiếp theo, kết quả được left join với dữ liệu thời tiết theo hai khóa site_id và timestamp. Việc sử dụng left join giúp giữ lại các bản ghi tiêu thụ năng lượng ngay cả khi một số thời điểm chưa có thông tin thời tiết tương ứng.

Tại tầng Silver, hệ thống cũng bổ sung các đặc trưng thời gian trực tiếp từ timestamp gồm hour, day_of_week, day_of_month, month và is_weekend. Trong đó, is_weekend nhận giá trị 1 khi ngày trong tuần là Chủ nhật hoặc Thứ bảy theo quy ước dayofweek của Spark, ngược lại nhận giá trị 0. Các đặc trưng này giúp dữ liệu Silver có thể được sử dụng thuận tiện hơn cho phân tích theo thời gian.

Trước khi ghi dữ liệu, chương trình thực hiện kiểm tra chất lượng bằng cách thống kê tổng số bản ghi, số bản ghi thiếu site_id sau join, số giá trị thiếu air_temperature và primary_use, đồng thời tính tổng số giá trị null cho các cột thời tiết. Kết quả Silver sau đó được ghi dưới dạng Parquet vào thư mục data/silver/ashrae_joined với chế độ overwrite.

Như vậy, tầng Silver đóng vai trò là lớp dữ liệu đã được làm sạch và tích hợp. So với tầng Bronze, dữ liệu tại đây đã liên kết thông tin tiêu thụ năng lượng, thuộc tính tòa nhà và điều kiện thời tiết trên cùng một bản ghi, đồng thời có thêm các biến thời gian phục vụ truy vấn và phân tích.

Các bước xử lý chính tại tầng Silver gồm:

- Đọc dữ liệu Parquet từ ba vùng Bronze.
- Chọn các cột nghiệp vụ cần thiết.
- Lọc các bản ghi thiếu khóa hoặc thiếu giá trị đo quan trọng.
- Loại bỏ dữ liệu trùng lặp theo khóa nghiệp vụ.
- Join dữ liệu năng lượng với metadata tòa nhà và dữ liệu thời tiết.
- Tạo các đặc trưng thời gian từ timestamp.
- Kiểm tra chất lượng dữ liệu sau join và ghi kết quả sang Parquet.

3.4.3. Tầng Gold

Tầng Gold được xây dựng từ dữ liệu Silver thông qua chương trình silver_to_gold.py. Đây là tầng có mức độ xử lý cao nhất trong pipeline hiện tại, tập trung vào xử lý giá trị thiếu, tạo thêm đặc trưng và lựa chọn bộ cột cuối cùng để phục vụ phân tích hoặc huấn luyện mô hình học máy.

Đầu tiên, chương trình kiểm tra các giá trị thiếu của year_built và floor_count. Giá trị trung vị của từng cột được tính bằng hàm percentile_approx ở mức 0,5. Nếu trung vị tồn tại, các giá trị null tương ứng được điền bằng trung vị. Riêng floor_count chỉ được giữ trong tập Gold nếu có thể tính được giá trị trung vị; nếu toàn bộ cột không có dữ liệu hữu ích, cột này sẽ bị loại khỏi danh sách đầu ra.

Đối với các biến thời tiết, hệ thống áp dụng chiến lược điền thiếu theo chuỗi thời gian. Hai cửa sổ được tạo theo site_id và sắp xếp theo timestamp. Với mỗi biến thời tiết, chương trình ưu tiên giữ giá trị hiện tại; nếu bị thiếu thì lấy giá trị gần nhất ở phía trước bằng forward fill, sau đó sử dụng giá trị gần nhất ở phía sau bằng backward fill. Sau bước này, hệ thống tiếp tục tính trung vị của từng biến thời tiết và dùng các giá trị trung vị làm phương án dự phòng cho những ô vẫn còn null.

Sau khi xử lý missing value, hệ thống thực hiện feature engineering. Biến meter_reading_log1p được tạo bằng phép biến đổi log1p trên meter_reading; square_feet_log1p được tạo tương tự từ square_feet. Tuổi tòa nhà building_age được

tính bằng năm của timestamp trừ year_built và chỉ giữ các giá trị không âm. Các đặc trưng chu kỳ hour_sin, hour_cos, month_sin và month_cos được tạo bằng hàm sine và cosine để biểu diễn tính tuần hoàn của giờ trong ngày và tháng trong năm. Ngoài ra, biến temperature_diff được tính bằng chênh lệch giữa air_temperature và dew_temperature.

Sau bước feature engineering, chương trình lựa chọn bộ cột Gold gồm khóa định danh, biến mục tiêu, thông tin tòa nhà, dữ liệu thời tiết, đặc trưng thời gian và các đặc trưng chu kỳ. Trước khi ghi dữ liệu, hệ thống hiển thị sample, schema, kiểm tra null trên các cột quan trọng, đếm tổng số bản ghi, thống kê min, max, mean, standard deviation của meter_reading, đồng thời kiểm tra phân phối theo meter và primary_use.

Dữ liệu Gold cuối cùng được ghi vào thư mục data/gold/ashrae_ml_ready dưới định dạng Parquet. Khác với Bronze và Silver, dữ liệu Gold được partition theo trường meter. Cách phân vùng này giúp tổ chức dữ liệu theo loại đồng hồ đo và có thể giảm lượng dữ liệu cần quét khi các tác vụ tiếp theo chỉ quan tâm đến một loại meter cụ thể.

Tóm lại, ba tầng lưu trữ tạo thành một chuỗi xử lý liên tục: Bronze chuẩn hóa và lưu dữ liệu gần với nguồn, Silver làm sạch và tích hợp các nguồn dữ liệu, còn Gold xử lý missing value và xây dựng đặc trưng để tạo tập dữ liệu sẵn sàng cho phân tích hoặc học máy. Trong code hiện tại, cả ba tầng đều được lưu bằng Apache Parquet; hệ thống chưa sử dụng Delta Lake hoặc các cơ chế transaction log của Delta Lake.

Bảng 3.x. So sánh vai trò của các tầng lưu trữ

Tầng	**Vai trò**	**Xử lý chính**	**Định dạng / vị trí**
Bronze	Lưu dữ liệu gần nguồn	Ép schema, chuẩn hóa timestamp, thêm metadata ingest	Parquet - data/bronze/...
Silver	Làm sạch và tích hợp	Lọc null, loại duplicate, join building + weather, tạo đặc trưng thời gian	Parquet - data/silver/ashrae_joined

Gold	Dữ liệu sẵn sàng phân tích/ML	Điền thiếu, log transform, building age, cyclical features, temperature_diff	Parquet partitionBy(meter) - data/gold/ashrae_ml_ready

3.5. Xây dựng mô hình dự báo tiêu thụ điện

3.5.1 Tập dữ liệu và cách chia theo thời gian

Pilot sử dụng điện RAW của BDG2 tại site Robin, chọn 10 tòa nhà theo thứ tự mã và 180 ngày đầu năm 2016. Tập này có 43.200 phép đo, tương ứng 4.320 giờ cho mỗi tòa nhà. Dữ liệu được chia theo các mốc giờ đã sắp xếp: 70% đầu cho train, 15% tiếp theo cho validation và 15% cuối cho test. Các tòa nhà tại cùng một giờ luôn thuộc cùng một tập.

Train được dùng để học thông số tiền xử lý và RF. Validation được dùng để tính residual và chọn ngưỡng. Test chỉ dùng để báo cáo các chỉ số cuối cùng và thử các kịch bản với model, ngưỡng đã cố định. Cách chia này tránh việc trộn ngẫu nhiên những giờ liền kề của cùng chuỗi vào cả train và test.

3.5.2. Đặc trưng và cấu hình huấn luyện

Bảng 3.5.1 Các nhóm đặc trưng của mô hình

Nhóm	Đầu vào	Xử lý
Phân loại	Tòa nhà, site, công năng	StringIndexer và OneHotEncoder
Thông tin tòa nhà	Diện tích, số tầng, năm xây dựng	log diện tích; median và cờ thiếu
Thời tiết	Nhiệt độ, điểm sương, mây, áp suất, gió	Median train và cờ thiếu
Lịch	Giờ, thứ, tháng, cuối tuần	sin và cos theo chu kỳ; cờ cuối tuần

Đặc trưng lịch được biểu diễn bằng cặp sin và cos để các vị trí ở cuối chu kỳ gần với đầu chu kỳ. Với c là vị trí trong chu kỳ và P là chu kỳ, công thức là:

$$c_{\sin} = \sin\left(\frac{2\pi c}{P}\right), \qquad c_{\cos} = \cos\left(\frac{2\pi c}{P}\right)$$

Chu kỳ giờ là 24, chu kỳ ngày trong tuần là 7 và chu kỳ tháng là 12. Diện tích được biến đổi bằng log(1+square_feet). Các biến số, cờ thiếu, đặc trưng lịch và biến phân loại đã mã hóa được ghép thành vector. Không sử dụng meter_reading hoặc residual làm đầu vào của RF.

Bảng 3.5.2 Cấu hình Random Forest trong pilot

Tham số	Giá trị
Số cây numTrees	60
Độ sâu tối đa maxDepth	8
Số bin tối đa maxBins	64
Số mẫu tối thiểu tại nút lá	3
Seed	42
Phân vị chọn ngưỡng	0,99
Số mẫu validation tối thiểu cho ngưỡng riêng	48
Phiên bản Spark	3.5.6

Đây là cấu hình khởi đầu đã được thử trên pilot, chưa phải cấu hình tối ưu qua tìm kiếm siêu tham số. Thời gian fit được đo riêng cho bước học Pipeline. Model được lưu cùng median, cấu hình đặc trưng và ngưỡng; khi scoring, dự đoán âm được giới hạn về 0 theo cùng quy tắc đã dùng lúc đánh giá.

3.6 Phát hiện bất thường

3.6.1 Tính mức tiêu thụ kỳ vọng và residual

Mỗi bản ghi hợp lệ được chuẩn hóa và tạo đặc trưng theo cấu hình train. RF sinh giá trị kỳ vọng, sau đó detector tính residual bằng actual trừ prediction và lấy độ lớn của

residual. Vì vector đặc trưng không chứa giá trị đo hiện tại, giá trị đo chỉ được dùng ở bước so sánh sau dự đoán.

3.6.2 Chọn ngưỡng và mức cảnh báo

Ngưỡng riêng từng tòa nhà được tạo trên validation bằng phân vị xấp xỉ 99% của độ lớn residual. Tòa nhà không đủ 48 mẫu hoặc chưa có ngưỡng riêng dùng ngưỡng chung. Ngưỡng được cố định trước đánh giá test và được nạp lại khi tích hợp streaming.

Bảng 3.6.1 Quy tắc gán mức cảnh báo

Mức	Điều kiện trên độ lớn residual
NORMAL	Không vượt ngưỡng
WARNING	Vượt ngưỡng và không vượt hai lần ngưỡng
CRITICAL	Vượt hai lần ngưỡng

3.6.3 Kịch bản kiểm tra detector

Các kịch bản gồm giữ nguyên, tăng vọt bằng 3y+1, giảm về 0 và lệch dương 20% bằng 1,2y. Với kịch bản tăng vọt, số cộng 1 mang cùng đơn vị kWh của mục tiêu. Mô hình và ngưỡng giữ nguyên; chỉ phép đo dùng để tính residual được sửa. Phép đo sửa nhưng vẫn bằng giá trị gốc không được gán là một thay đổi mới.

Đánh giá dùng cùng tập test cho các bản sao theo kịch bản. Nhãn chỉ nguồn thay đổi đã tạo, vì dữ liệu gốc chưa được xác nhận bình thường. Kết quả phải nêu nguồn nhãn, độ mạnh và tần suất kịch bản để người đọc hiểu giới hạn của Precision, Recall và F1.

3.6.4 Cấu trúc đầu ra và vận hành

Đầu ra gồm building_id, site_id, meter, timestamp, meter_reading, prediction, residual, abs_residual, threshold, is_anomaly và severity. Nếu bản ghi có event_id, run_id và produced_at_ms, các trường này được giữ để truy vết. Hàm scoring xử lý micro-batch; việc ghi sink cần theo cơ chế phù hợp để một event_id không bị tạo lại khi Spark retry. Trước khi huấn luyện lại, cần theo dõi sai số theo từng tòa nhà và tỷ lệ bản ghi thiếu đặc trưng. Thay đổi mạnh của phân phối residual có thể do chế độ vận hành mới hoặc chất lượng đầu vào. Model, median và ngưỡng phải được cập nhật như một bộ nhất quán; mỗi phiên bản cần đánh giá trên khoảng thời gian độc lập trước khi đưa vào vận hành.

3.7 Thiết kế kịch bản kiểm thử và chỉ số đánh giá hệ thống

3.7.1 Kiểm thử chất lượng phân tích

Nhóm kiểm thử chất lượng gồm đánh giá hồi quy trên test theo thời gian, đối chiếu với đối chứng lịch và kiểm tra detector bằng các phép đo sửa có chủ đích. MAE, RMSE và R^2 được tính trên toàn test; biểu đồ xem trước phải ghi rõ số dòng dùng để vẽ. Các kịch bản detector sử dụng model và ngưỡng đã cố định, không điều chỉnh ngưỡng sau khi quan sát test.

3.7.2 Kiểm thử theo tải và dữ liệu lỗi

Phép thử tải của pilot dùng 100, 500 và 1.000 message/s, mỗi mức lặp ba lần và giữ nguyên cấu hình máy, số partition, batch size theo byte, trigger và cách ghi telemetry. Kết quả chín lượt được trình bày ở mục 4.8. Đây là mức tải replay của Producer, không thay đổi độ phân giải một giờ của BDG2.

Mỗi lượt có run_id riêng. Warm-up 30 giây được phát và xử lý hết trước tập đo; sau đó phát event đo trong 60 giây và chờ sink ghi đủ trước khi kết thúc cửa sổ quan sát. Warm-up được loại khỏi kết quả tổng hợp. Backlog được ghi lúc dừng phát và sau drain để phân biệt tồn đọng theo trigger với khả năng xử lý bù.

Các ca bổ sung gồm timestamp lỗi, giá trị âm, trường bắt buộc thiếu, trùng khóa, metadata hoặc weather thiếu, dữ liệu đến trễ, tòa nhà chưa gặp và retry micro-batch. Bản ghi loại bỏ ở upstream phải được thống kê riêng để tránh diễn giải sự giảm số đầu ra thành Kafka lag

3.7.3 Quy ước đo và thu thập log

Mỗi event đo có event_id ổn định và produced_at_ms ghi ngay trước khi gửi. Điểm cuối ghi ready_at_ms sau khi sink đã ghi thành công cùng event_id. Nếu điểm cuối là giao diện, cần thêm dấu thời gian khi API hoặc trình duyệt quan sát bản ghi đó. Đồng hồ giữa các máy phải được đồng bộ hoặc phải nêu cách đo trên một đồng hồ chung. Spark progress được thu cho đúng query scoring, gồm numInputRows, inputRowsPerSecond, processedRowsPerSecond và durationMs.triggerExecution [4]. Không cộng progress của nhiều query đang cùng đọc một nguồn. Trong callback có nhiều action, input được cache và materialize một lần để hạn chế việc nguồn bị tính lặp khi lấy các chỉ số. Receipts retry được loại trùng theo event_id trước tổng hợp latency và delivered throughput.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Dữ liệu và cấu hình phát thử nghiệm

Nguồn phát là tập dữ liệu ASHRAE đã qua tầng Silver. Bốn tệp Parquet trong thư mục này có tổng cộng 20.216.100 bản ghi theo metadata của tệp. Mỗi bản ghi kết hợp số đo công tơ với mã tòa nhà, thông tin sử dụng tòa nhà và dữ liệu thời tiết. Python Producer đọc lần lượt từng tệp Parquet theo lô 2.048 dòng để tránh nạp toàn bộ tập dữ liệu vào bộ nhớ. Theo cấu hình mặc định của scripts/producer.py, Producer chỉ phát loại công tơ meter = 0 (điện năng), gửi tối đa 5.000 bản tin ở tốc độ mục tiêu 100 bản tin/giây. Khi đánh giá tải, các tham số --rate, --batch-size và --limit cho phép thay đổi tốc độ mục tiêu, chu kỳ xả bộ đệm và số bản tin cần phát. Giá trị --rate 0 tắt bước chờ giữa các bản tin. Tham số --batch-size của script là số bản tin giữa hai lần gọi flush(); bộ đệm nội bộ của KafkaProducer được cấu hình riêng ở mức 16.384 byte.

Thành phần	Cấu hình
Nguồn dữ liệu	- Silver Parquet: data/silver/ashrae_joined - 4 tệp và 20.216.100 bản ghi
Broker	- Apache Kafka 4.2.1, KRaft, chạy cục bộ - bootstrap localhost:9092
Topic	- energy_raw - script tạo topic yêu cầu 3 partition, replication factor = 1
Khóa bản tin	- building_id được mã hóa thành chuỗi UTF-8
Giá trị bản tin	- JSON mã hóa UTF-8 - gồm số đo, thời gian, thông tin tòa nhà và thời tiết
Producer	- acks=1; retries=3 - batch_size nội bộ=16.384 byte; linger_ms=10
Tham số chạy mặc định	--meter 0; --rate 100; --batch-size 500; --limit 5000

Khóa building_id định tuyến các bản tin của cùng một tòa nhà vào cùng một partition, nhờ đó Kafka duy trì thứ tự ghi của chúng trong partition đó. Topic energy_raw được cấu hình ba partition để phân phối dữ liệu và hỗ trợ xử lý song song. Broker KRaft hoạt động độc lập với replication factor bằng 1, phù hợp với môi trường thực nghiệm của nhóm.

4.2. Kết quả ETL và lưu trữ dữ liệu

4.2. Kết quả ETL và lưu trữ dữ liệu

Phần này trình bày kết quả thực nghiệm của pipeline ETL trên bộ dữ liệu ASHRAE, từ dữ liệu CSV ban đầu đến các tầng Bronze, Silver và Gold. Các số liệu được ưu tiên lấy từ log chạy Spark thực tế của hệ thống. Đối với những chỉ tiêu chưa có file đầu ra hoặc chưa có log benchmark trong môi trường hiện tại, báo cáo ghi rõ trạng thái chưa đo thay vì suy diễn thành kết quả thực nghiệm.

Phạm vi số liệu: Kết quả RAW → Bronze và các thống kê chất lượng tại Silver đã có log thực nghiệm. Trong bộ dữ liệu hiện được cung cấp cho báo cáo chưa có thư mục Parquet đầu ra Gold cũng như log benchmark tách riêng thời gian đọc, ghi và xử lý; vì vậy các phần tương ứng được trình bày theo trạng thái xác nhận thực tế.

4.2.1. Kết quả chuyển dữ liệu RAW → Bronze

Pipeline RAW → Bronze được thực thi bằng Apache Spark 4.2.0. Ba tập dữ liệu đầu vào gồm train.csv, building_metadata.csv và weather_train.csv được đọc bằng schema khai báo tường minh. Trong quá trình chuyển đổi, timestamp của dữ liệu tiêu thụ năng lượng và thời tiết được chuyển sang kiểu Timestamp, đồng thời bổ sung hai trường metadata _ingested_at và _source_file để phục vụ truy vết nguồn dữ liệu.

Bảng 4.1. Kết quả chuyển đổi từ RAW sang Bronze

Nguồn dữ liệu	Số bản ghi	Null timestamp	Kiểm tra khác	Đầu ra Bronze
train.csv	20.216.100	0	0 meter_reading	data/bronze/train
building_metadata.csv	1.449	Không áp dụng	Không ghi nhận trong log	data/bronze/building_metadata
weather_train.csv	139.773	0	Không áp dụng	data/bronze/weather_train

Kết quả cho thấy toàn bộ 20.216.100 bản ghi tiêu thụ năng lượng, 1.449 bản ghi metadata tòa nhà và 139.773 bản ghi thời tiết đã được đọc thành công. Hai trường timestamp được kiểm tra đều không có giá trị null, trong khi meter_reading của bảng

train cũng không có giá trị thiếu. Điều này cho thấy dữ liệu đầu vào đáp ứng được các điều kiện tối thiểu để chuyển sang bước làm sạch và tích hợp ở tầng Silver.

Sau khi kiểm tra, ba DataFrame được ghi xuống Parquet tại các vùng Bronze tương ứng. Log chạy trên môi trường WSL xác nhận bước ghi hoàn tất. Trong quá trình ghi tập train lớn, Spark phát cảnh báo áp lực bộ nhớ khi tổng cấp phát vượt khoảng 95% heap; tuy nhiên quá trình ghi vẫn hoàn thành. Đây là dấu hiệu cần lưu ý khi mở rộng pipeline trên máy có giới hạn RAM.

4.2.2. Kết quả làm sạch Bronze → Silver

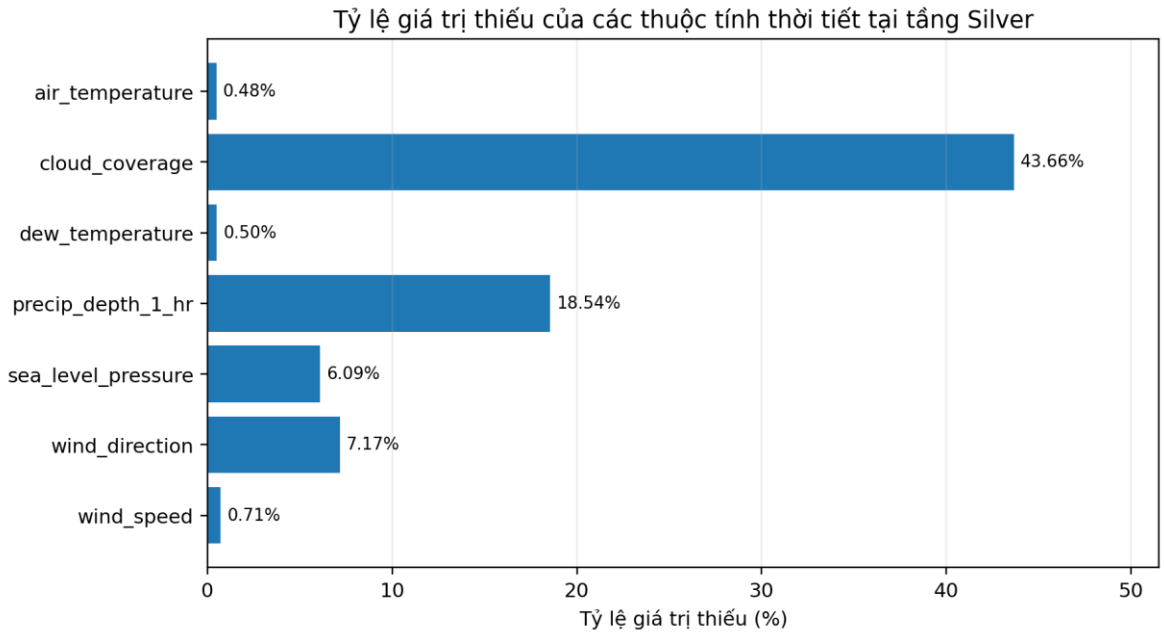
Ở tầng Silver, hệ thống loại bỏ các bản ghi thiếu khóa hoặc thiếu meter_reading, xóa bản ghi trùng lặp theo khóa nghiệp vụ, sau đó tích hợp dữ liệu theo hai bước. Bảng train được broadcast join với building_metadata theo building_id và tiếp tục left join với dữ liệu thời tiết theo cặp site_id, timestamp. Sau join, năm thuộc tính thời gian hour, day_of_week, day_of_month, month và is_weekend được tạo thêm.

Bảng 4.2. Một số chỉ tiêu chất lượng dữ liệu sau Bronze → Silver

Chỉ tiêu	**Số lượng**	**Tỷ lệ trên Silver**
Tổng số bản ghi Silver	20.216.100	100,00%
Thiếu liên kết building (site_id null)	0	0,00%
Thiếu primary_use	0	0,00%
Thiếu air_temperature	96.658	0,48%

Bảng 4.3. Giá trị thiếu của các thuộc tính thời tiết tại Silver

Thuộc tính	**Số giá trị thiếu**	**Tỷ lệ (%)**
air_temperature	96.658	0,48%
cloud_coverage	8.825.365	43,66%
dew_temperature	100.140	0,50%
precip_depth_1_hr	3.749.023	18,54%
sea_level_pressure	1.231.669	6,09%
wind_direction	1.449.048	7,17%
wind_speed	143.676	0,71%



Hình 4.1. Tỷ lệ giá trị thiếu của các thuộc tính thời tiết tại tầng Silver

Số lượng bản ghi Silver vẫn giữ ở mức 20.216.100, cho thấy các phép làm sạch khóa chính và loại trùng không làm giảm số dòng của tập train trong lần chạy được ghi nhận. Đồng thời, không có bản ghi bị thiếu liên kết building và không có primary_use bị thiếu sau join. Điều này xác nhận phép nối giữa train và building_metadata hoạt động đầy đủ đối với dữ liệu đang sử dụng. Ngược lại, dữ liệu thời tiết vẫn còn thiếu ở nhiều mức độ khác nhau. cloud_coverage có tỷ lệ thiếu cao nhất, khoảng 43,66%, tiếp theo là precip_depth_1_hr với khoảng 18,54%. Các trường air_temperature và dew_temperature chỉ thiếu dưới 0,5%. Vì vậy, các giá trị thời tiết không nên bị loại bỏ trực tiếp theo dòng; thay vào đó pipeline Gold sử dụng forward fill, backward fill và median fallback để giảm mất mát dữ liệu.

4.2.3. Kết quả tổng hợp Silver → Gold

Phiên bản hiện tại của pipeline Silver → Gold đã hoàn thiện logic xử lý dữ liệu thiếu và feature engineering. Đối với year_built và floor_count, hệ thống tính trung vị bằng percentile_approx và chỉ thực hiện fill khi trung vị tồn tại. Đối với dữ liệu thời tiết, giá trị thiếu được điền theo thứ tự: giá trị hiện tại, giá trị gần nhất phía trước, giá trị gần nhất phía sau và cuối cùng là trung vị toàn cột.

Sau bước xử lý thiếu, hệ thống tạo các đặc trưng mới gồm meter_reading_log1p, square_feet_log1p, building_age, hour_sin, hour_cos, month_sin, month_cos và

temperature_diff. Tập Gold cuối cùng được chọn lại các cột cần thiết và ghi dưới dạng Parquet, phân vùng theo meter tại data/gold/ashrae_ml_ready.

Bảng 4.4. Trạng thái tổng hợp dữ liệu Silver → Gold

Hạng mục	**Kết quả / cấu hình**
Đầu vào Silver	20.216.100 dòng
Xử lý year_built	Median fill nếu có trung vị
Xử lý floor_count	Bỏ qua fill/loại khỏi Gold nếu toàn bộ null
Xử lý weather null	Forward fill → backward fill → median fallback
Feature engineering	8 nhóm đặc trưng mới
Số dòng Gold cuối cùng	Kỳ vọng giữ nguyên 20.216.100
Gold Parquet	data/gold/ashrae_ml_ready, partitionBy(meter)

4.2.4. So sánh dung lượng CSV và Parquet

Mục tiêu của phép so sánh là đánh giá hiệu quả lưu trữ khi chuyển dữ liệu nguồn dạng CSV sang Parquet. CSV là định dạng văn bản theo hàng và lặp lại biểu diễn chuỗi cho từng giá trị, trong khi Parquet lưu dữ liệu theo cột và hỗ trợ nén, do đó thường phù hợp hơn với truy vấn phân tích trên Spark.

Bảng 4.5. Trạng thái đo dung lượng lưu trữ

Định dạng / tầng	**Phạm vi đo**
RAW CSV	train.csv + building_metadata.csv + weather_train.csv
Bronze Parquet	data/bronze/*
Silver Parquet	data/silver/ashrae_joined
Gold Parquet	data/gold/ashrae_ml_ready

Do các thư mục dữ liệu thực tế không nằm trong gói file đang dùng để tạo báo cáo, không thể đưa ra con số MB/GB đáng tin cậy cho tỷ lệ nén CSV → Parquet ở lần đánh giá này. Việc tự đặt một tỷ lệ giảm dung lượng sẽ làm sai bản chất thực nghiệm vì dung lượng Parquet còn phụ thuộc codec nén, schema, số partition và cách Spark ghi file.

- Dung lượng CSV: tổng kích thước ba file trong data/raw.
- Dung lượng Bronze: tổng kích thước các file part-*.parquet trong data/bronze.
- Tỷ lệ giảm dung lượng = (Dung lượng CSV - Dung lượng Parquet) / Dung lượng CSV × 100%.
- Cần đo trên cùng một bộ dữ liệu và cùng máy/ổ đĩa để phép so sánh có ý nghĩa.

4.2.5. So sánh thời gian đọc, ghi và xử lý ETL

Để đánh giá hiệu năng ETL chính xác, cần tách thời gian đọc dữ liệu, thời gian biến đổi và thời gian ghi kết quả. Với Spark, cần đặc biệt lưu ý cơ chế lazy evaluation: lệnh spark.read hoặc các phép withColumn/join chỉ xây dựng execution plan; thời gian thực sự chỉ phát sinh khi có action như count(), show() hoặc write. Vì vậy không thể dùng thời gian tạo DataFrame làm thời gian đọc thực tế.

Bảng 4.6. Trạng thái đo thời gian xử lý ETL

Hạng mục	Thời gian	Ghi chú
RAW → Bronze (toàn giai đoạn)	≈ 53 giây	Ước tính từ mốc log 16:55:37 đến khoảng 16:56:30; không phải timer benchmark

Mốc khoảng 53 giây của RAW → Bronze chỉ nên được sử dụng như số tham khảo về thời gian chạy toàn giai đoạn trong lần thực thi WSL đã ghi log. Không nên tách con số này thành thời gian đọc và ghi vì Spark thực thi lười và nhiều action kiểm tra chất lượng cũng kích hoạt job riêng.

Để hoàn thiện benchmark, mỗi phép đo nên được lặp lại ít nhất ba lần sau một lần warm-up, ghi lại giá trị trung bình và độ lệch chuẩn. Khi so sánh CSV với Parquet, cần dùng cùng schema, cùng tập cột và cùng action. Đối với thời gian ghi, nên xóa output cũ hoặc ghi sang đường dẫn mới trước mỗi lần chạy để tránh ảnh hưởng của trạng thái file trước đó. Nhìn chung, kết quả hiện có xác nhận pipeline đã xử lý thành công dữ liệu RAW và xây dựng được tầng Silver với hơn 20 triệu bản ghi. Các thống kê null cũng cho thấy nhu cầu xử lý missing value tại tầng Gold là cần thiết, đặc biệt đối với cloud_coverage và precip_depth_1_hr. Để kết luận định lượng về hiệu quả lưu trữ và

tốc độ xử lý, cần bổ sung phép đo trực tiếp trên các thư mục CSV/Parquet đầu ra của chính lần chạy cuối cùng.

4.3. Kết quả Kafka và Python Producer

4.3.1. Định dạng bản tin và kiểm tra topic

Producer chuyển mỗi dòng Parquet thành một sự kiện JSON với 13 trường: building_id, site_id, meter, timestamp, meter_reading, primary_use, square_feet, floor_count, air_temperature, dew_temperature, cloud_coverage, sea_level_pressure và wind_speed. Giá trị timestamp được chuẩn hóa thành chuỗi YYYY-MM-DD HH:mm:ss; các trường thời tiết hoặc số tầng có thể nhận null. Script có thể đọc topic energy_raw và in partition, offset, building_id, timestamp và meter_reading để kiểm tra trực quan luồng bản tin. Một dòng đầu vào thực tế trong Silver có building_id = 0, site_id = 0, meter = 0, timestamp = 2016-01-01 06:00:00, meter_reading = 0, primary_use = Education và square_feet = 7432; floor_count là null. Đây là ví dụ lấy từ Parquet nguồn để minh họa dữ liệu Producer sẽ đóng gói. Thư mục log Kafka hiện có các partition energy_raw-0, energy_raw-1 và energy_raw-2, phù hợp với cấu hình ba partition.

4.3.2. Thông lượng Producer theo tốc độ phát

Phép thử phát cùng 1.000 bản ghi đầu vào ở mỗi kịch bản. Để so sánh tốc độ mục tiêu, hai lượt dùng 200 và 500 msg/s với ba partition, flush sau mỗi 500 bản tin. Để so sánh batch size, ba lượt không giới hạn tốc độ sử dụng flush sau 100, 500 và 1.000 bản tin. Lượt cuối giữ flush 500 và dùng topic một partition. Thời gian đo tính từ khi bắt đầu gọi send() đến sau flush() và xác nhận của broker.

Partition	Tốc độ đặt (msg/s)	Flush mỗi (bản tin)	Broker xác nhận / Consumer đọc	Thông lượng (msg/s)
3	200	500	1000 / 1000	178,83
3	500	500	1000 / 1000	382,89
3	0	100	1000 / 1000	5048,14
3	0	500	1000 / 1000	5063,1
3	0	1000	1000 / 1000	5020,93
1	0	500	1000 / 1000	5084,92

Khi tăng tốc độ mục tiêu từ 200 lên 500 msg/s, thông lượng thực tế tăng từ 178,83 lên 382,89 msg/s. Ở chế độ không giới hạn tốc độ, Producer đạt 5.020,93–5.084,92 msg/s trên cùng tập 1.000 bản ghi. Kết quả cho thấy Producer đáp ứng các mức tải đã khảo sát và có thể điều chỉnh tốc độ phát theo yêu cầu thử nghiệm.

4.3.3. Ảnh hưởng của batch size và số partition

Với topic ba partition và chế độ phát không giới hạn, các mức flush 100, 500 và 1.000 bản tin lần lượt đạt 5.048,14; 5.063,10 và 5.020,93 msg/s. Khi giữ flush 500, topic một partition đạt 5.084,92 msg/s và topic ba partition đạt 5.063,10 msg/s. Cả hai cấu hình partition và cả ba mức flush đều duy trì thông lượng trên 5.000 msg/s trong phép đo. Trong báo cáo này, --batch-size biểu thị số bản tin giữa hai lần flush(); bộ đệm nội bộ của KafkaProducer được cấu hình riêng.

4.3.4. Độ ổn định và tỷ lệ gửi thành công

Ở cả sáu kịch bản, Producer phát 1.000 bản tin mỗi lượt; broker xác nhận đủ 1.000 bản tin và Consumer đọc lại đủ 1.000 bản tin theo khoảng offset tương ứng. Không ghi nhận lỗi gửi. Tỷ lệ xác nhận và tỷ lệ đọc lại đạt 100% ở mọi cấu hình đã thử. Việc đối chiếu offset trước và sau từng lượt cũng xác nhận số bản tin của mỗi kịch bản được ghi nhận đầy đủ trên topic. Kết quả cho thấy đường truyền Producer → Kafka → Consumer vận hành ổn định trong các phép thử của nhóm.

4.5 Kết quả mô hình dự báo tiêu thụ điện

4.5.1 Kết quả huấn luyện Random Forest Regression

Thực nghiệm phần AI được chạy ngày 04/10/2026 bằng Python 3.12, Java 17 và PySpark 3.5.6. Dữ liệu gồm 10 tòa nhà tại site Robin, từ 00:00 ngày 01/01/2016 đến 23:00 ngày 28/06/2016. Bộ chuẩn bị đọc CSV điện RAW, metadata và weather chính thức. Trong 43.200 phép đo được chọn, không có phép đo âm hoặc không hữu hạn và không có dòng trùng bị loại. Có 170 dòng không ghép được bản ghi weather cùng giờ; các trường thiếu được xử lý theo cấu hình train.

Bảng 4.5.1 Chia dữ liệu theo thời gian

Tập	Số dòng	Khoảng thời gian theo giờ địa phương
Train	30.240	01/01/2016 00:00 đến 05/05/2016 23:00

Tập	Số dòng	Khoảng thời gian theo giờ địa phương
Validation	6.480	06/05/2016 00:00 đến 01/06/2016 23:00
Test	6.480	02/06/2016 00:00 đến 28/06/2016 23:00

RF dùng 60 cây, độ sâu tối đa 8, maxBins=64, minInstancesPerNode=3 và seed=42. Thời gian fit Pipeline là 8,319 giây ở môi trường thử nghiệm. Model, ngưỡng, dữ liệu held-out và các file đánh giá đã được lưu. Toàn bộ 6.480 dự đoán test và đối chứng được đối chiếu bằng phép tính độc lập; các chỉ số khớp với bảng kết quả [5].

4.5.2 Đánh giá bằng MAE, RMSE và R^2

Bảng 4.5.2 Kết quả hồi quy gộp trên các tòa nhà

Mô hình và tập	N	MAE kWh	RMSE kWh	R^2
RF validation	6.480	21,2968	32,3284	0,9737
RF test	6.480	22,2301	31,7544	0,9760
Đối chứng test	6.480	23,0557	32,5963	0,9747

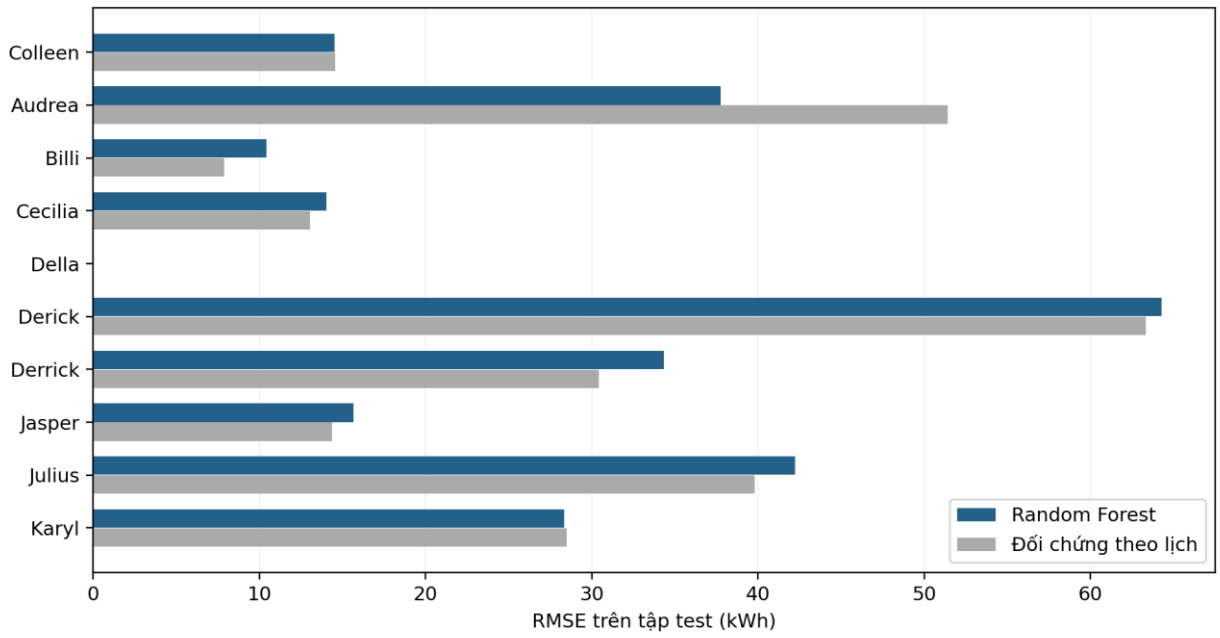
Trên test, RF có MAE 22,2301 kWh và RMSE 31,7544 kWh. Đối chứng có RMSE 32,5963 kWh; RF giảm RMSE khoảng 2,58%. Đây là mức cải thiện nhỏ trên chỉ số gộp. R^2 của RF đạt 0,9760 nhưng còn chịu ảnh hưởng bởi sự khác biệt quy mô tiêu thụ giữa các tòa nhà. Cần đánh giá riêng từng chuỗi để biết model có theo được biến động tại từng tòa nhà hay không.

Bảng 4.5.3 Kết quả riêng từng tòa nhà tại Robin

Tòa nhà	MAE RF	RMSE RF	RMSE đối chứng	R^2 RF
Colleen	12,102	14,536	14,565	-1,6571
Audrea	34,938	37,730	51,407	-3,5175
Billi	8,620	10,426	7,877	0,7635
Cecilia	10,352	14,046	13,059	0,2240

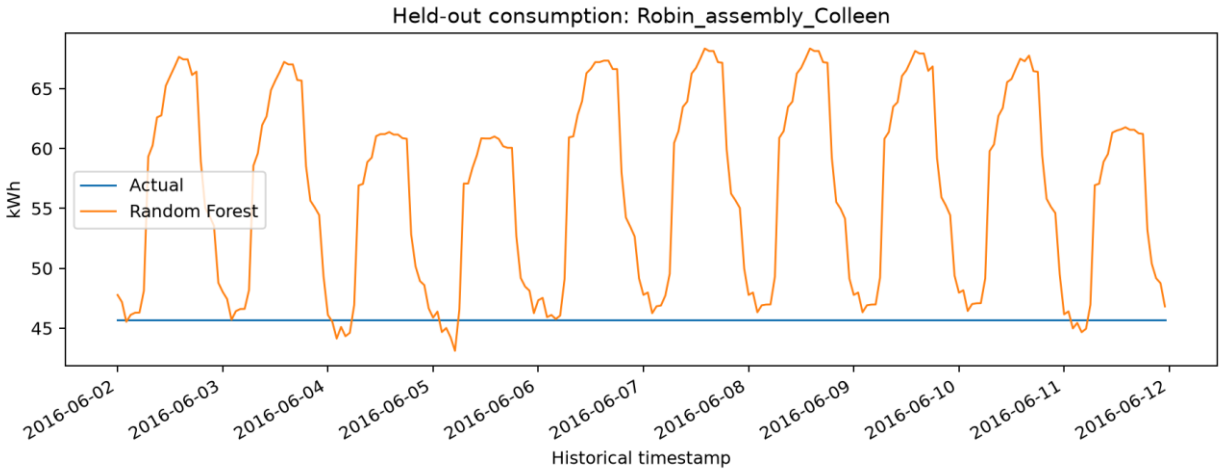
Tòa nhà	MAE RF	RMSE RF	RMSE đối chứng	R^2 RF
Della	0,0000214	0,0001910	0,002	Không xác định
Derick	52,795	64,285	63,326	0,5065
Derrick	26,438	34,329	30,417	0,7202
Jasper	12,933	15,657	14,386	0,5327
Julius	35,983	42,221	39,789	0,8399
Karyl	28,141	28,351	28,501	Không xác định

Mỗi tòa nhà có 648 dòng test; các mã rút gọn trong bảng tương ứng mã gốc Robin_assembly_Colleen và Robin_education_ cùng tên tòa nhà. RF có RMSE nhỏ hơn đối chứng ở 4/10 tòa nhà. Colleen và Audrea có R^2 âm; Della và Karyl có giá trị test không đổi nên R^2 không xác định. Kết quả này cho thấy chỉ số gộp cao chưa đủ để khẳng định RF phù hợp cho mọi tòa nhà. Với từng chuỗi, có thể cần mô hình hoặc cấu hình riêng, nhưng phải chọn trên train và validation trước khi đánh giá lại test.



Hình 4.5.1 So sánh RMSE test của RF và đối chứng theo từng tòa nhà

4.5.3 So sánh giá trị dự báo với giá trị thực tế

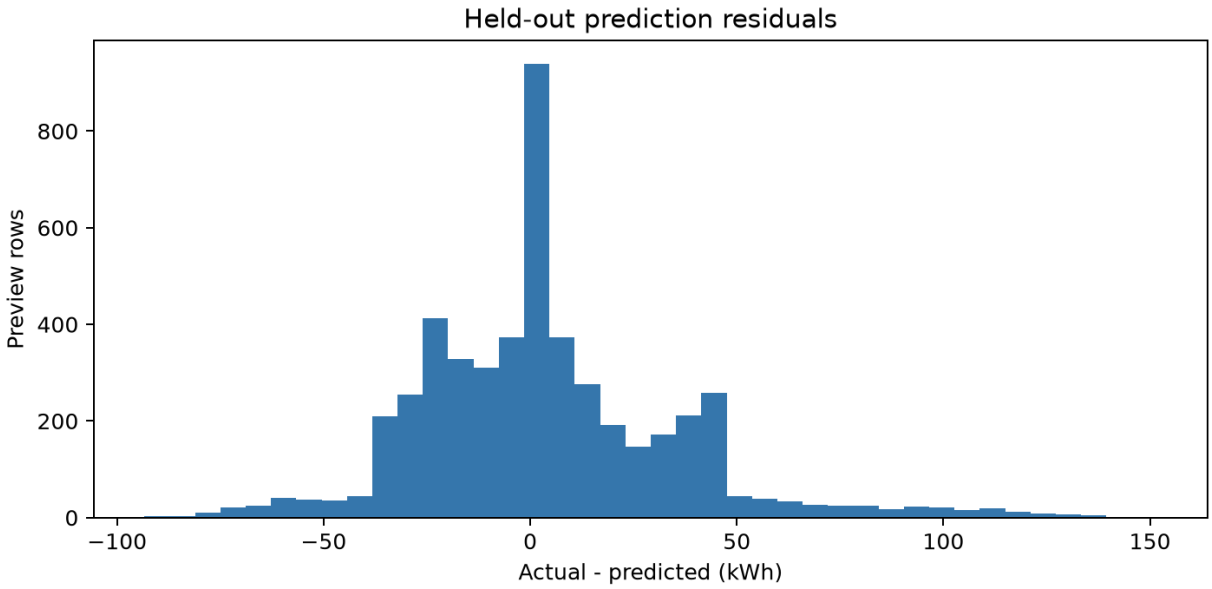


Hình 4.5.2 Phép đo và RF tại Robin_assembly_Colleen trong 240 giờ đầu test

Trong đoạn được vẽ, phép đo tại Colleen gần như giữ nguyên, còn RF tạo dao động theo lịch. Đường ước lượng có nhiều giờ lớn hơn phép đo, thể hiện sai lệch ngay cả khi chỉ số gộp của toàn bộ các tòa nhà còn cao. Đây là trường hợp cần kiểm tra lịch vận hành và chất lượng dữ liệu. Không có nhãn độc lập để kết luận đoạn gần như không đổi là cảm biến hỏng hay một chế độ tiêu thụ thực tế.

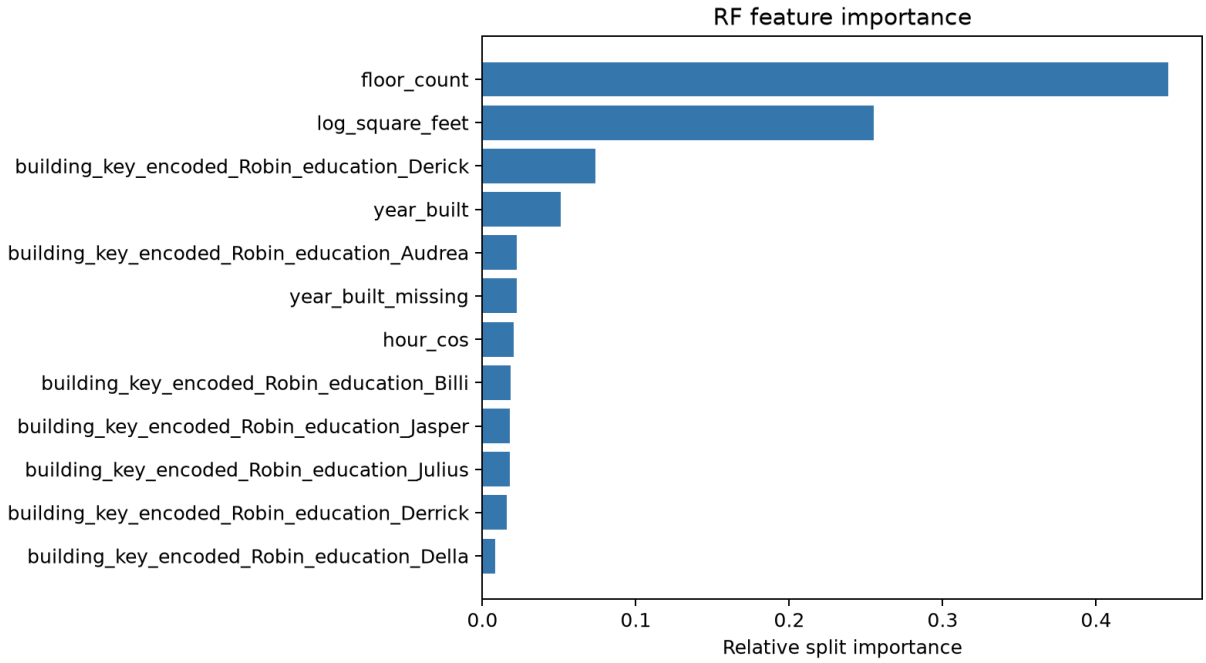
Biểu đồ lấy tòa nhà đầu tiên theo thứ tự mã trong mẫu xem trước, không chọn theo mức sai số tốt nhất. Bộ dữ liệu dự đoán đầy đủ gồm 6.480 dòng, còn file preview và biểu đồ phân bố được giới hạn tối đa 5.000 dòng. Giới hạn này chỉ dùng để đọc và trực quan hóa, không thay đổi tập dùng tính MAE, RMSE và R^2.

4.5.4 Phân tích sai số và đặc trưng quan trọng



Hình 4.5.3 Phân bố residual trên 5.000 dòng xem trước của test

Phân bố residual có các mẫu ở cả hai phía của 0 và có đuôi sai số lớn. Residual dương cho biết điện năng đo được lớn hơn ước lượng; residual âm cho biết ước lượng lớn hơn phép đo. Các sai số lớn làm RMSE cao hơn MAE và ảnh hưởng đến ngưỡng cảnh báo riêng của tòa nhà. Việc kiểm tra cần kết hợp từng chuỗi và tình trạng thiếu weather, thay vì chỉ đọc histogram gộp.



Hình 4.5.4 Đặc trưng quan trọng của Random Forest trong pilot

Ba đặc trưng có importance cao nhất là floor_count với 0,4468; log_square_feet với 0,2551; và biến mã hóa của Robin_education_Derick với 0,0737. Chúng giúp model phân biệt mức tiêu thụ giữa các tòa nhà. Các biến quy mô và mã tòa nhà chiếm tỷ trọng lớn, phù hợp với việc chỉ số gộp phản ánh cả khác biệt giữa tòa nhà. Importance mô tả mức sử dụng đặc trưng trong các bước chia cây, không chứng minh tác động nhân quả. Các đặc trưng có tương quan cũng có thể chia sẻ importance.

4.6 Kết quả phát hiện bất thường

4.6.1 Kết quả tính residual và ngưỡng bất thường

Tất cả 10 tòa nhà có 648 mẫu validation nên đều đủ điều kiện tạo ngưỡng riêng. Ngưỡng được tính trước khi sửa các phép đo test. Ngưỡng chung khi không có ngưỡng riêng là 111,1564 kWh; trong test của pilot, các mã tòa nhà đều đã có ngưỡng riêng.

Bảng 4.6.1 Ngưỡng riêng theo tòa nhà

Tòa nhà tại Robin	Ngưỡng kWh
Colleen	29,3038
Audrea	102,8066
Billi	23,0837
Cecilia	37,6926
Della	0,0000528
Derick	167,5121
Derrick	94,2944
Jasper	34,9330
Julius	135,5181
Karyl	33,4605

Ngưỡng Della rất nhỏ do residual validation gần 0; ngưỡng Derick đạt 167,5121 kWh. Chênh lệch ngưỡng cho thấy một mức lệch cố định theo kWh không có ý nghĩa giống nhau ở mọi tòa nhà. Với chuỗi ít biến động, cần cân nhắc ngưỡng tối thiểu gắn với độ

phân giải công tơ trong các nghiên cứu tiếp theo; pilot này giữ nguyên ngưỡng đã chọn để tránh điều chỉnh sau khi nhìn test.

4.6.2 Các kịch bản bất thường được thử nghiệm

Từ 6.480 bản ghi test, chương trình tạo các bản sao giữ nguyên, tăng vọt, giảm về 0 và lệch dương 20%. Mỗi kịch bản có 6.480 mẫu trong pilot này. Các kịch bản sửa có tổng cộng 19.440 mẫu dương theo định nghĩa nhãn; nhóm giữ nguyên có 6.480 mẫu tham chiếu. Tổng số mẫu đánh giá có kiểm soát là 25.920.

Bảng 4.6.2 Số cảnh báo theo kịch bản

Kịch bản	Số mẫu	Số cảnh báo	Tỷ lệ cảnh báo
Giữ nguyên	6.480	106	1,64%
Tăng vọt 3y+1	6.480	5.641	87,05%
Giảm về 0	6.480	5.896	90,99%
Lệch dương 20%	6.480	2.448	37,78%

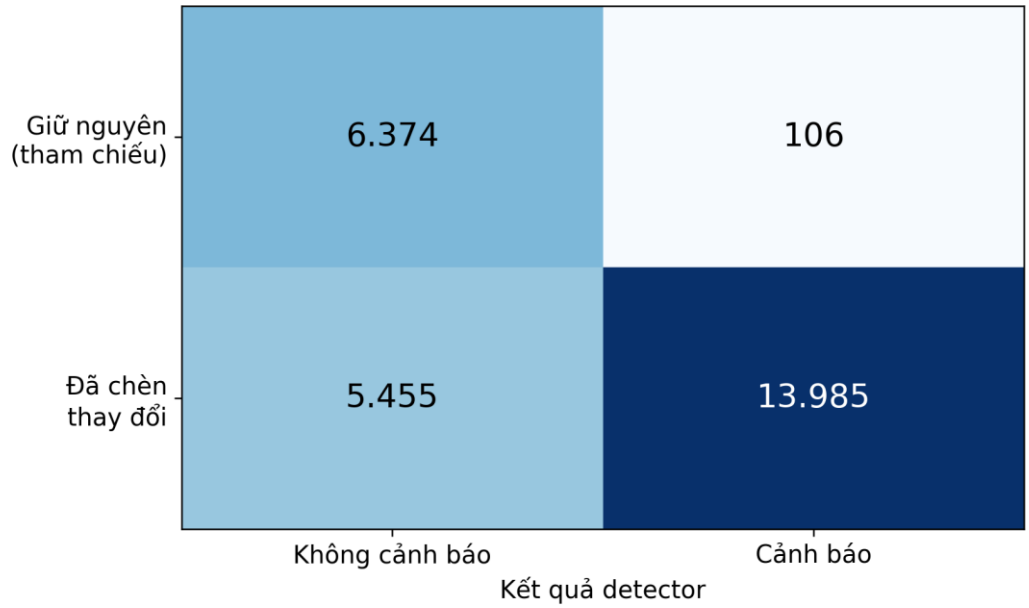
Các tỷ lệ trong bảng là tỷ lệ cảnh báo theo từng kịch bản, chưa phải tỷ lệ sự cố ngoài thực tế. Kịch bản giảm về 0 và tăng vọt làm sai lệch lớn hơn nên được phát hiện nhiều hơn lệch dương 20%. Nhóm giữ nguyên có 106 cảnh báo; các bản ghi này chưa được xác nhận bình thường nên chưa đủ căn cứ gọi chúng là báo động giả thực tế.

4.6.3 Đánh giá bằng Precision, Recall và F1 score

Bảng 4.6.3 Kết quả theo nhãn chèn thay đổi có kiểm soát

Chỉ số	Giá trị
TP	13.985
FP theo nhãn tham chiếu	106
FN	5.455
TN	6.374
Precision	0,9925

Chỉ số	Giá trị
Recall	0,7194
F1 score	0,8342



Hình 4.6.1 Ma trận nhầm lẫn theo nhãn thay đổi đã chèn có kiểm soát

Precision đạt 0,9925 nghĩa là phần lớn cảnh báo trong tập thí nghiệm nằm trên các phép đo đã sửa. Recall đạt 0,7194 cho thấy còn khoảng 28,06% phép đo sửa không được cảnh báo. F1 đạt 0,8342. Kết quả phụ thuộc các mức sửa 3y+1, 0 và 1,2y cũng như tỷ lệ ba nhóm sửa so với nhóm giữ nguyên. Chưa thể dùng các chỉ số này để kết luận model phát hiện đúng sự cố vận hành thực tế.

4.6.4 Phân tích các trường hợp phát hiện đúng và sai

Phép đo sửa được cảnh báo: tại Robin_education_Audrea, giờ 2016-06-02 00:00:00, sau khi tăng 20%, phép đo là 410,5124 kWh, mức kỳ vọng 301,0550 kWh; độ lớn residual 109,4575 kWh so với ngưỡng 102,8066 kWh, vượt ngưỡng nên được cảnh báo.

Phép đo sửa bị bỏ sót: tại Robin_assembly_Colleen, giờ 2016-06-02 00:00:00, sau khi tăng 20%, phép đo là 54,7914 kWh, mức kỳ vọng 47,7720 kWh; độ lớn residual 7,0194 kWh so với ngưỡng 29,3038 kWh, không vượt ngưỡng nên không được cảnh báo.

Phép đo gốc được cảnh báo: tại Robin_education_Jasper, giờ 2016-06-02 10:00:00, phép đo gốc là 226,4000 kWh, mức kỳ vọng 190,1386 kWh; độ lớn residual 36,2614 kWh so với ngưỡng 34,9330 kWh, vượt ngưỡng nên được cảnh báo.

Kịch bản lệch dương 20% chỉ tạo 2.448 cảnh báo trong 6.480 mẫu, tương ứng 37,78%; đây là hạn chế rõ của độ nhạy với thay đổi nhỏ. Ngưỡng cao có thể che khuất một mức tăng nhẹ, còn ngưỡng thấp ở chuỗi gần như không đổi có thể nhạy với sai số rất nhỏ. Các nhận xét này là cơ sở cho thiết kế thí nghiệm tiếp theo, chưa được dùng để sửa ngưỡng và công bố lại cùng test.

Bộ scoring đã sinh các trường cảnh báo trên test, gồm actual, prediction, residual, threshold, is_anomaly và severity. Bảng dưới minh họa một số đầu ra từ phép đo thật giữ nguyên. Đây là dữ liệu để nối vào dashboard; các bản ghi được cảnh báo chưa có nhãn xác nhận sự cố.

Bảng 4.7.1 Ví dụ đầu ra cảnh báo trên test thật

Tòa nhà và giờ	Actual	Prediction	Residual	Ngưỡng	Mức
Jasper 06-02 10:00	226,400	190,139	36,261	34,9330	WARNING
Karyl 06-02 11:00	0,007	33,676	-33,669	33,4605	WARNING
Della 06-03 00:00	0,749	0,749	0,0003366	0,0000528	CRITICAL

Các cột điện năng trong bảng có đơn vị kWh; thời gian là giờ địa phương của site Robin. Với Della, cần đọc ngưỡng nhỏ với đủ chữ số để tránh hiểu nhầm khi làm tròn. Khi đưa lên dashboard, nên cho phép xem chi tiết thay vì chỉ hiển thị nhãn mức độ.

Trong benchmark, các trường cảnh báo đã được ghi thật vào MongoDB cùng event_id và run_id. File mongo_alert_examples.json chứa các ví dụ truy vấn từ collection sau khi chạy; mongo_scored_sample.json chứa mẫu cả bản ghi bình thường và cảnh báo. Việc đã ghi sink cung cấp dữ liệu cho giao diện nhưng chưa chứng minh dashboard của nhóm đã hiển thị chúng.

Bổ sung khi tích hợp Chèn ảnh dashboard thật thể hiện một bản ghi trong bảng hoặc một event_id đã ghi sink, kèm tòa nhà, thời gian, giá trị đo, prediction, residual, threshold và severity. Chưa có ảnh hoặc log chứng minh dashboard của nhóm đã hiển thị đầu ra pilot.

4.6.5 Minh họa quy trình giám sát hoàn chỉnh

Pilot đã thực hiện đường Producer gửi JSON vào Kafka, Spark đọc và làm sạch, ghép đặc trưng, RF scoring, detector gán mức và MongoDB ghi kết quả. Một ví dụ dưới đây được truy vết bằng event_id giữa document, receipt và progress của batch. ACK Producer được kiểm tra bằng tổng số của lượt; bộ log hiện không ghi ACK theo partition và offset cho từng event.

Event được minh họa: phuc_20261004T131302Z_68cc49_r1000_n1:00000139. Run_id: phuc_20261004T131302Z_68cc49_r1000_n1.

Bảng 4.7.2 Bản ghi đã đi qua luồng xử lý pilot

Trường	Giá trị thực ghi
Tòa nhà và giờ đo	Robin_education_Karyl 2016-06-02 13:00:00
Actual và prediction kWh	0,0070 và 33,6923
Residual và ngưỡng kWh	-33,6853 và 33,4605
Mức cảnh báo và batch	WARNING; batch_id=128
Produced epoch ms	1791120238264
MongoDB ACK epoch ms	1791120241707
Độ trễ đến sink ms	3443

Timestamp 2016 là giờ quan sát của dữ liệu BDG2; các dấu epoch ms là thời điểm replay ngày 04/10/2026 và dùng để tính latency. File event_trace.json giữ document, receipt, thời gian các phần callback và Spark progress liên quan. Chuỗi bằng chứng này kết thúc ở MongoDB.

Bổ sung cho dashboard Theo dõi cùng event_id qua API và ảnh giao diện thật, ghi thời điểm bản ghi xuất hiện. Các bằng chứng deployment của nhóm được phối hợp với Hùng, Dương và Dung để hoàn thành minh họa đến giao diện.

4.8 Đánh giá hiệu năng toàn hệ thống

Benchmark ngày 04/10/2026 đã đo luồng Producer phát dữ liệu BDG2 thật qua Kafka, Spark Structured Streaming và MongoDB. Spark thực hiện parse JSON, ghép metadata và weather tĩnh, tạo đặc trưng, RF scoring và gán cảnh báo residual. Điểm cuối là ACK sau khi MongoDB ghi có journal. Toàn bộ dịch vụ chạy trên một máy; đây là pilot độc lập, chưa phải phép đo deployment hoặc dashboard của nhóm [9].

4.8.1 Benchmark theo nhiều mức tải

Bảng 4.8.1 Cấu hình thực chạy benchmark

Thành phần	Cấu hình
Máy và CPU	Linux x86_64; quota 8 core; 9 logical CPU quan sát
Giới hạn RAM	8 GiB theo cgroup
Runtime	Python 3.12.14; Java 17; Spark 3.5.6
Kafka	3.9.1 KRaft; một broker; 3 partition; replication=1; heap tối đa 512 MiB
Spark	local[2]; driver 2 GiB; shuffle 4 partition; trigger 5 giây
MongoDB	7.0.18 standalone; cache WiredTiger 256 MiB; w=1, j=true; mỗi chunk 500 document
Producer	acks=all; idempotence; linger 5 ms; batch.size=65.536 byte; không nén

Model RF đã lưu dùng 60 cây, depth 8; metadata và weather lịch sử tĩnh được cache, không huấn luyện lại trong benchmark. Kafka chạy một broker và controller KRaft cùng máy [7]. Không đặt maxOffsetsPerTrigger. Mỗi lượt warm-up 30 giây được xử lý hết trước khi phát tập đo 60 giây. Các mức tải được chạy lần lượt 100, 500 và 1.000

message/s, mỗi mức ba lượt, với cấu hình giữ nguyên. Telemetry được bật trong toàn bộ phép đo nên chi phí thu log có nằm trong kết quả.

Bảng 4.8.2 Số sự kiện trong các lượt đo

Tải msg/s	Lượt lặp	Event mỗi lượt	Event đo tổng	Event warm up
100	3	6.000	18.000	9.000
500	3	30.000	90.000	45.000
1.000	3	60.000	180.000	90.000

Tổng cộng 288.000 event đo và 144.000 event warm-up đã được gửi và ghi sink. Nguồn replay là 6.480 phép đo test thật, đọc tuần tự và lặp lại khi hết; mỗi lần phát được gán event_id mới. Lượt 100 message/s dùng 6.000 event, còn các mức cao có nhiều vòng replay hơn. Số event tải không biểu thị 288.000 quan sát lịch sử khác nhau.

Đối chiếu từng lượt cho thấy số ACK Producer, số dòng Spark xử lý, số receipt duy nhất và số document MongoDB đều bằng số event dự kiến; không có dòng invalid hoặc duplicate bị loại trong tập đo. Backlog cuối cùng bằng 0. Trước khi đo, 40 prediction streaming được so với reference offline và có sai lệch tuyệt đối lớn nhất bằng 0. File analysis_verification.json giữ đầy đủ kiểm tra này [9].

4.8.2 Throughput toàn hệ thống

Producer throughput và delivered throughput được tách theo điểm đo:

$$R_{\text{producer}} = \frac{N_{\text{ACK}}}{T_{\text{send}}}, \qquad R_{\text{delivered}} = \frac{N_{\text{ready,unique}}}{T_{\text{observation}}}$$

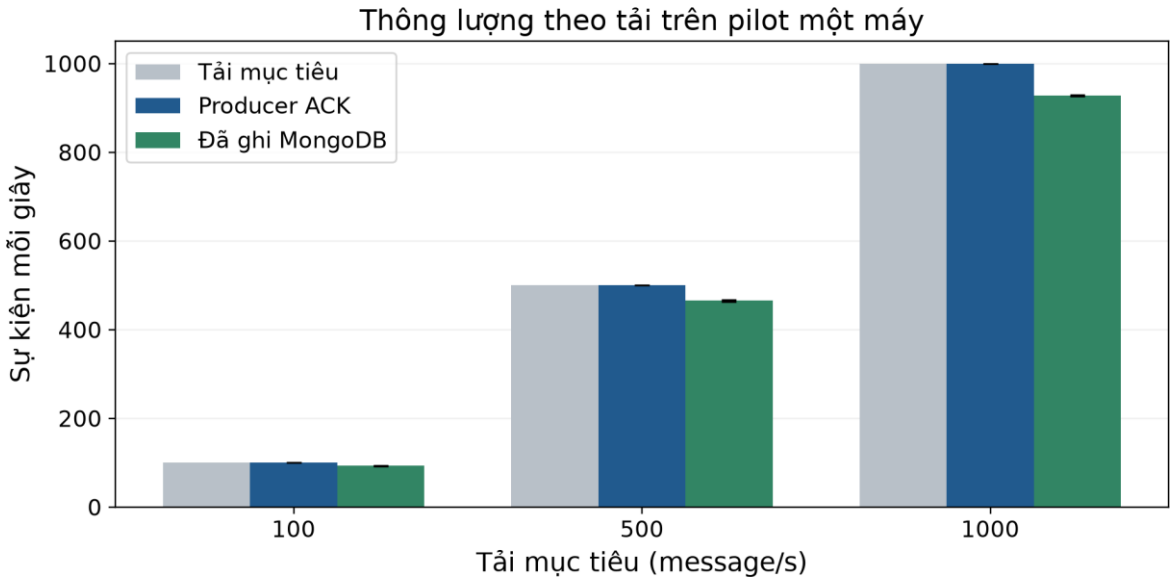
Thông lượng Producer dùng số message ACK thành công chia thời gian phát gồm flush ACK. Thông lượng delivered dùng event_id duy nhất đã ghi MongoDB chia cửa sổ từ bắt đầu phát đến ACK sink cuối cùng, gồm thời gian chờ trigger và drain. Mỗi hàng dưới là trung bình và độ lệch chuẩn mẫu của ba lượt; tất cả lượt được giữ trong benchmark_summary.csv.

Khả năng xử lý Spark được tính riêng bằng tổng numInputRows chia tổng durationMs.triggerExecution của các batch có dữ liệu trong một lượt [4]. Đây là tốc độ trên thời gian trigger hoạt động, không phải thông lượng giao kết quả theo đồng hồ thực.

inputRowsPerSecond và processedRowsPerSecond nguyên gốc cũng được lưu để đối chiếu.

Bảng 4.8.3 Thông lượng trung bình và SD của ba lượt

Tải msg/s	Producer ACK/s	Delivered event/s	Spark active row/s
100	100,00 ± 0,00	92,89 ± 0,88	270,93 ± 14,56
500	499,99 ± 0,00	465,12 ± 1,92	1265,71 ± 90,71
1.000	999,98 ± 0,01	927,89 ± 1,83	2264,22 ± 212,57



Hình 4.8.1 Thông lượng đo thực theo tải với SD giữa ba lượt

Ở mức 1.000 message/s, Producer đạt trung bình 999,98 ACK/s; thông lượng ghi MongoDB đạt 927,89 event/s khi tính cả drain. Sự khác biệt giữa hai điểm đo phản ánh cửa sổ delivered dài hơn cửa sổ phát. Do đã đối chiếu số lượng bằng nhau, sự khác biệt này không biểu thị mất event.

4.8.3 End to End Latency

Độ trễ của event i được xác định theo hai dấu thời gian trên cùng event_id:

$$L_i = t_{i,\text{ready}} - t_{i,\text{produced}}$$

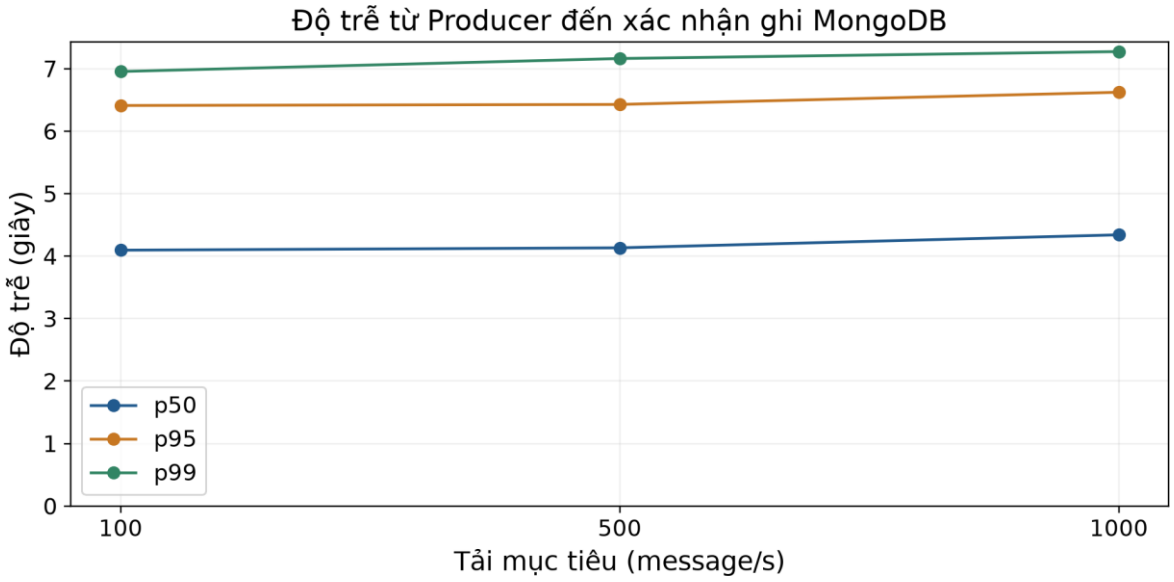
produced_at_ms được ghi ngay trước khi gửi JSON; ready_at_ms ghi sau khi MongoDB ACK bulk upsert với w=1, j=true. Trong cấu hình này, ACK được chờ sau khi ghi vào journal trên đĩa [8]. Timestamp lịch sử 2016 không dùng tính độ trễ replay. Hai dấu đo

dùng cùng đồng hồ máy; độ phân giải lưu log là millisecond. Phép đo chưa gồm API hoặc thời điểm dashboard hiển thị.

Bảng 4.8.4 Độ trễ Producer đến MongoDB theo mili giây

Tải msg/s	Mean ms	p50 ms	p95 ms	p99 ms
100	4107,1	4094,5	6410,0	6954,0
500	4139,8	4132,0	6427,1	7162,0
1.000	4323,5	4340,0	6622,0	7273,0

Mean và các phân vị được tính bằng cách gộp mọi receipt event đo duy nhất của ba lượt cùng mức tải, dùng nội suy tuyến tính. Số event tương ứng là 18.000, 90.000 và 180.000; warm-up bị loại theo run_id. Không có receipt âm, thiếu hoặc bị loại. Đây là phân vị gộp theo event, không phải trung bình của ba p95.



Hình 4.8.2 Phân vị độ trễ đến MongoDB trên toàn event đo

Ở mức tải cao nhất, p95 là 6,622 giây và p99 là 7,273 giây. Trigger 5 giây làm event có thể phải chờ micro-batch; thời gian parse, enrichment, scoring và ghi journal tiếp tục cộng vào thời gian chờ. Muốn định lượng ảnh hưởng riêng của trigger, cần phép so sánh thêm với cấu hình trigger khác.

4.8.4 Backlog và khả năng bắt kịp dữ liệu

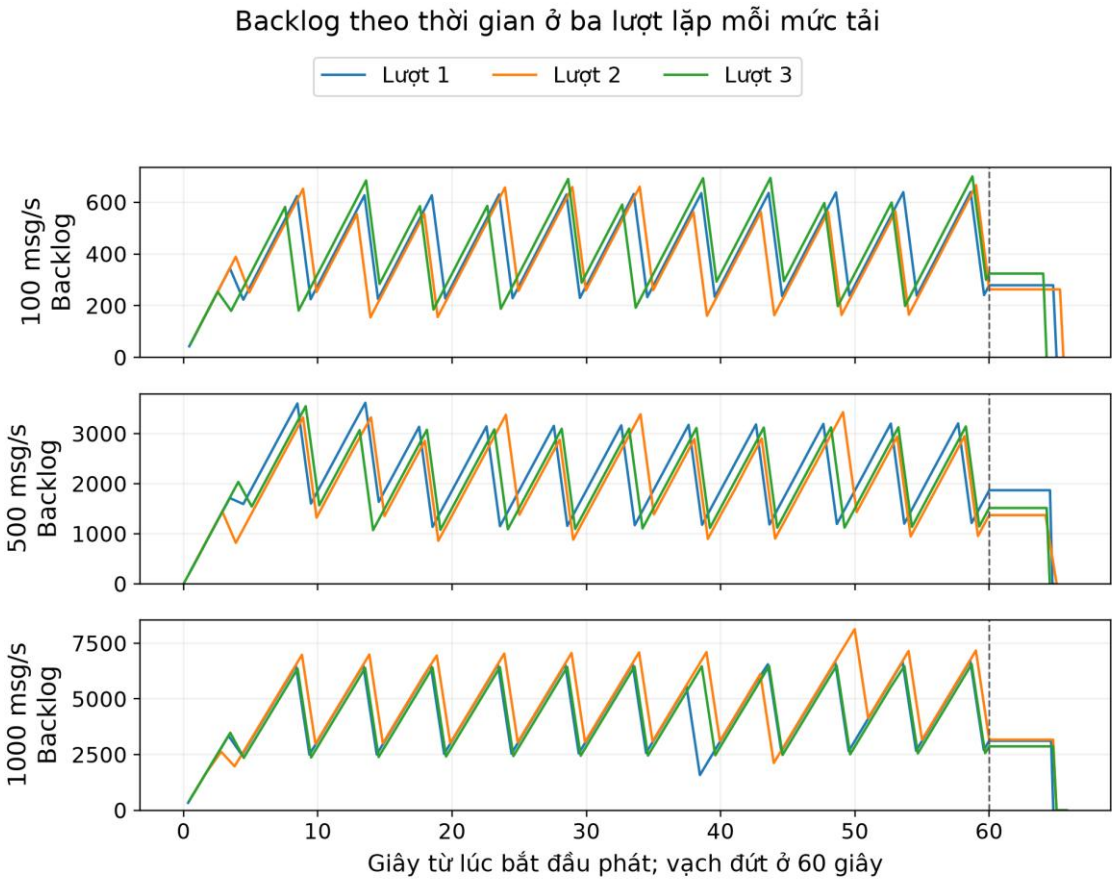
Backlog cộng chênh lệch Kafka latest offset và endOffset của micro-batch Spark đã hoàn thành, theo từng partition. Cả hai dùng quy ước vị trí message tiếp theo:

$$\text{Backlog}(t) = \sum_{p} [o_{p,\text{latest}}(t) - o_{p,\text{processed}}(t)]$$

Telemetry đọc khoảng mỗi giây và thêm snapshot khi dừng phát, sau drain. Vị trí Spark được chụp trước khi truy vấn latest offset để tránh số âm do thứ tự quan sát. Đây là phép lấy mẫu gần đồng thời, không phải snapshot nguyên tử. Spark Kafka source không commit offset theo consumer group, nên không lấy group lag thay thế vị trí xử lý [6]. Batch chỉ được coi là hoàn thành sau callback ghi sink.

Bảng 4.8.5 Backlog và thời gian drain của ba lượt

Tải msg/s	Cuối phát TB ± SD	Đỉnh mẫu min đến max	Sau drain	Drain giây TB ± SD
100	288,7 ± 31,6	640 đến 700	0	4,597 ± 0,613
500	1585,3 ± 255,9	3.428 đến 3.616	0	4,499 ± 0,267
1.000	3051,0 ± 160,5	6.560 đến 8.133	0	4,661 ± 0,127



Hình 4.8.3 Backlog lấy mẫu theo thời gian ở đủ chín lượt

Đỉnh backlog là đỉnh đã quan sát theo mẫu, không bảo đảm là cực đại tuyệt đối giữa hai lần đọc. Drain đo từ lúc Producer kết thúc gửi và flush ACK đến ACK MongoDB cuối cùng; kiểm tra kết thúc còn yêu cầu offset Spark bắt kịp và số document MongoDB khớp. Ở tải 1.000 message/s, drain trung bình 4,661 giây; cả ba lượt sau drain đều bằng 0.

Backlog dao động theo chu kỳ micro-batch là phù hợp với việc nguồn tiếp tục nhận event trong lúc Spark chờ hoặc xử lý. Việc về 0 sau drain xác nhận toàn bộ tập event đã được xử lý trong các cửa sổ đã thử. Kết quả 60 giây mỗi lượt chưa thay thế thử nghiệm tải kéo dài hàng giờ hoặc chứng minh hàng đợi luôn thấp khi nguồn phát liên tục.

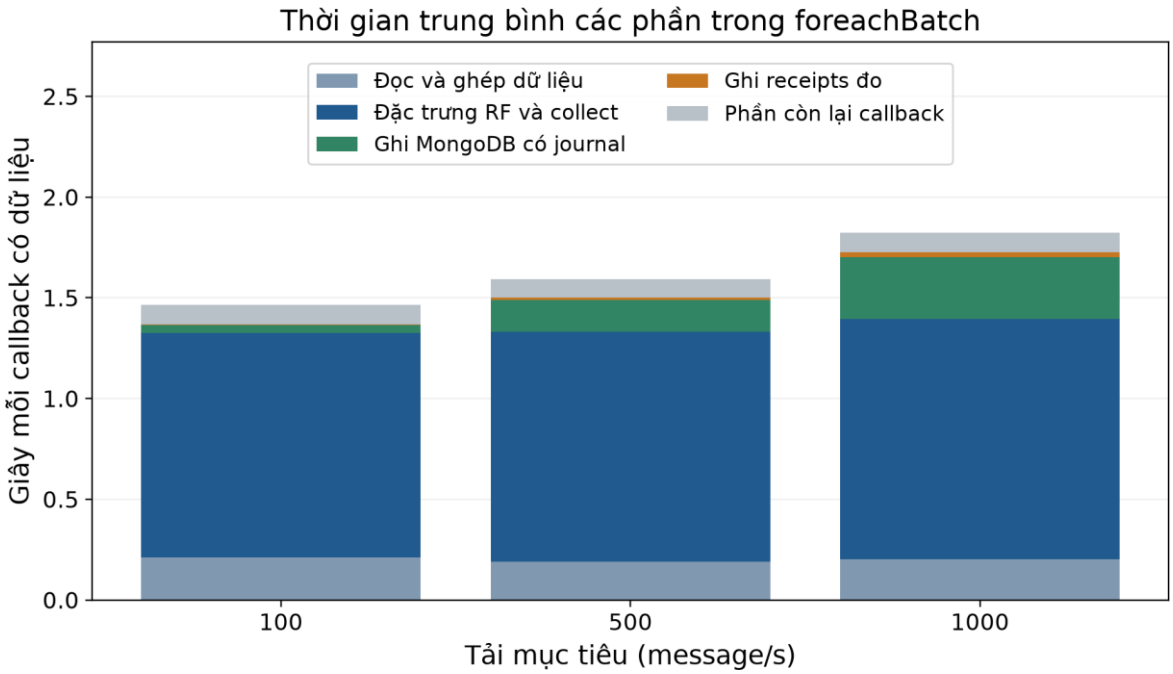
4.8.5 Điểm nghẽn, khả năng mở rộng và nhận xét chung

Bảng 4.8.6 Thời gian trung bình mỗi callback có dữ liệu

Tải msg/s	Đọc và ghép ms	Score collect ms	Ghi Mongo ms	Callback ms
100	212,5	1113,9	38,3	1465,6

Tải msg/s	Đọc và ghép ms	Score collect ms	Ghi Mongo ms	Callback ms
500	188,9	1143,9	157,2	1591,3
1.000	202,4	1193,3	306,6	1823,5

Đọc và ghép bao gồm materialize input đã parse và broadcast join. Score collect bao gồm tạo đặc trưng, chạy Pipeline RF, detector và đưa kết quả về Python driver; không thể diễn giải riêng thành thời gian inference RF. Ghi Mongo là tổng thời gian các bulk upsert chờ journal ACK. Callback còn có tìm run_id, tạo document, ghi receipts và phần điều phối, nên tổng ba cột thành phần không bằng toàn bộ callback.



Hình 4.8.4 Thời gian các phần trong foreachBatch theo tải

Trong ba phần được đo ở tải 1.000 message/s, phần tạo đặc trưng, RF, detector và collect có thời gian trung bình lớn nhất. Đây là quan sát về phân bố thời gian callback ở cấu hình đã chạy, chưa chứng minh thành phần đó là giới hạn tuyệt đối của toàn hệ thống. File offset_resources.jsonl còn giữ RSS và CPU theo process; các số này không được cộng rồi gọi là tài nguyên toàn máy vì chưa bao gồm đầy đủ child worker.

Pilot đã nhận và ghi đủ event ở cả ba mức tải, nhưng chưa thử tải trên 1.000 message/s, mở rộng ngang hoặc đổi số core. Kafka replication=1 và MongoDB standalone cũng chưa kiểm tra chịu lỗi. Không công bố một mức tải tối đa từ ba mức đã thử. Thử tiếp có

thể thay từng yếu tố như core Spark, partition, trigger hoặc cách ghi sink rồi chạy lại cùng dữ liệu và đủ các lượt lặp.

Giới hạn phạm vi gồm metadata và weather lịch sử tĩnh đã cache, replay tuần tự 6.480 dòng test, các dịch vụ cùng máy, micro-batch ghi qua Python driver và thời gian phát 60 giây. Đường đo chưa gồm Delta ETL, Airflow, huấn luyện lại, mạng giữa máy, API hoặc dashboard. Các kết quả này cung cấp bằng chứng hiệu năng cho phần pilot của Phúc; khi đánh giá triển khai nhóm cần chạy lại trên code và cấu hình triển khai thực.

KẾT LUẬN

Đề tài đã xây dựng và triển khai thành công một hệ thống xử lý dữ liệu lớn theo thời gian thực, áp dụng kiến trúc phân tầng Bronze - Silver - Gold thông qua Apache Spark kết hợp cùng nền tảng truyền tải Apache Kafka. Thực nghiệm cho thấy đường ống dữ liệu (data pipeline) hoạt động cực kỳ ổn định và đáng tin cậy với khả năng chịu tải lên tới 1.000 thông điệp/giây mà không ghi nhận bất kỳ sự cố lỗi gửi hay mất mát dữ liệu nào, đạt tỷ lệ xác nhận 100%. Khả năng xử lý luồng mạnh mẽ của hệ thống còn được minh chứng qua việc giải quyết triệt để lượng dữ liệu tồn đọng (backlog giảm về 0 sau khi drain kết thúc) và duy trì độ trễ toàn trình (end-to-end latency) từ lúc phát dữ liệu Kafka đến khi ghi nhận thành công vào cơ sở dữ liệu MongoDB ở mức rất thấp, với phân vị p99 chỉ ở khoảng 7,273 giây ở mức tải cao nhất.

Đối với bài toán phân tích và dự báo nhu cầu tiêu thụ năng lượng, đề tài đã tích hợp hiệu quả mô hình Random Forest Regression thông qua thư viện Spark MLlib. Đánh giá gộp trên tập dữ liệu kiểm thử cho thấy mô hình học máy đạt hiệu năng khả quan với hệ số xác định R2 đạt mức 0,9760, tạo ra một đường tham chiếu độ chính xác cao cho mức tiêu thụ kỳ vọng. Tuy nhiên, qua quá trình phân tích chi tiết các chỉ số sai số (MAE và RMSE), nghiên cứu cũng làm rõ được tính chất không đồng nhất giữa các công trình; sự chênh lệch lớn về mức độ biến động tiêu thụ khẳng định rằng hệ thống cần hướng tới việc tùy chỉnh mô hình và thiết lập cấu hình riêng biệt cho đặc thù của từng tòa nhà, thay vì áp dụng một bộ tham số dự báo chung. Trên cơ sở kết quả ước lượng từ mô hình, hệ thống đã hoàn thiện cơ chế phát hiện bất thường tự động dựa trên độ lớn sai số dự báo (residual), sử dụng ngưỡng phân vị 99% được xác định từ tập validation. Khi được kiểm thử bằng các kịch bản bất thường có kiểm soát, cơ chế này thể hiện độ chuẩn xác (Precision) ấn tượng đạt 0,9925 và khả năng phát hiện (Recall) đạt 0,7194, đặc biệt nhạy bén với các sự cố có độ chênh lệch tiêu thụ rõ rệt như tăng vọt hoặc giảm đột ngột về 0, dù vẫn còn không gian để tối ưu hóa đối với các thay đổi nhỏ lẻ (như lệch 20%). Toàn bộ kết quả tính toán, dự báo và mức độ cảnh báo đều được luân chuyển xuyên suốt, tự động hóa hoàn toàn và ghi nhận thành công vào MongoDB, tạo nên một chu trình khép kín sẵn sàng phục vụ cho việc tích hợp, giám sát trực quan trên giao diện Dashboard của tòa nhà thông minh.

TÀI LIỆU THAM KHẢO

[1] C. Miller, et al., 'The Building Data Genome Project 2, energy meter data from the ASHRAE Great Energy Predictor III competition', Scientific Data, vol. 7, no. 1, p. 368, 2020.

[2] J. Kreps, N. Narkhede, and J. Rao, 'Kafka: A distributed messaging system for log processing', Proc. 6th Int. Workshop Netw. Meet. Databases (NetDB), 2011, pp. 1-7.

[3] J. Kreps, 'Questioning the Lambda Architecture', O'Reilly Media, 2014. Available: https://www.oreilly.com/radar/questioning-the-lambda-architecture/

[4] Apache Kafka Documentation, 'KIP-500: Replace ZooKeeper with a Self-Managed Metadata Quorum', Apache Software Foundation, 2020. Available: https://cwiki.apache.org/confluence/display/KAFKA/KIP-500

[5] Apache Kafka Documentation, 'Kafka KRaft Mode and Storage Tooling', Apache Software Foundation, 2024. Available: https://kafka.apache.org/documentation/#kraft